



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A MINOR PROJECT REPORT
ON
EDGE-AI BASED PREDICTIVE CLOSED CONTROL SYSTEM FOR SMART
POULTRY FARMING**

Submitted By:

Amir Bhattarai	(THA080BEI005)
Aviral Adhikari	(THA080BEI011)
Balram Sharma Kandel	(THA080BEI012)
Bikash Bk	(THA080BEI015)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

August 2026



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A MINOR PROJECT REPORT
ON
EDGE-AI BASED PREDICTIVE CLOSED CONTROL SYSTEM FOR SMART
POULTRY FARMING**

Submitted By:

Amir Bhattarai	(THA080BEI005)
Aviral Adhikari	(THA080BEI011)
Balram Sharma Kandel	(THA080BEI012)
Bikash Bk	(THA080BEI015)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

In partial fulfillment for the award of the
Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Anup Shrestha

August 2026

DECLARATION

We hereby declare that the project entitled, “**EDGE-AI BASED PREDICTIVE CLOSED CONTROL SYSTEM FOR SMART POULTRY FARMING**” in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Electronics, Communication and Information Engineering** is a bona fide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Amir Bhattarai (THA080BEI005) _____

Aviral Adhikari (THA080BEI011) _____

Balram Sharma Kandel (THA080BEI012) _____

Bikash Bk (THA080BEI015) _____

Date: August 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a minor project work entitled **“EDGE-AI BASED PREDICTIVE CLOSED CONTROL SYSTEM FOR SMART POULTRY FARMING”** submitted by **Amir Bhattarai, Aviral Adhikari, Balram Sharma Kandel, Bikash Bk** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found that the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's Degree of Electronics, Communication, and Information Engineering.

Project Supervisor
Er. Anup Shrestha
Department of Electronics and Computer
Engineering
Thapathali Campus, IOE

External Examiner

Project Coordinator
Er. Praches Acharya
Department of Electronics and Computer
Engineering
Thapathali Campus, IOE

Head of Department
Er. Umesh Kanta Ghimire
Department of Electronics and Computer
Engineering
Thapathali Campus, IOE

August 2026

COPYRIGHT

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

We would like to thank Institute of Engineering, Tribhuvan University, for incorporating major project within the curriculum of Bachelor's in Electronics, Communication, and Information Engineering.

Special thanks to our supervisor, **Er. Anup Shrestha**, for his guidance, invaluable feedback, and constant support for the project. His expertise, experience in domain knowledge of Scheduling and encouragement have been pivotal in shaping this project.

We would like to express our appreciation to the **Department of Electronics and Computer Engineering**, for their continuous assistance, recommendations during project selection. This has significantly helped in enabling our professional development.

We extend our sincere thanks to all the teachers, friends, and both direct and indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Amir Bhattarai	(THA080BEI005)
Aviral Adhikari	(THA080BEI011)
Balram Sharma Kandel	(THA080BEI012)
Bikash Bk	(THA080BEI015)

ABSTRACT

Industrial poultry farming relies on raising chickens in an artificial controlled environment that must provide all the conditions. Critical environmental factors like temperature and ammonia must remain within safe limits as unfavourable conditions can negatively affect bird growth so these factors should be well monitored and controlled. Traditional monitoring systems are reactive and would alert the administrator only after the threshold for the danger level exceeds. This project presents a distributed Edge-AI architecture that makes the poultry management autonomous and includes predictive intervention for the problems occurring in poultry farms. The hardware architecture consists of a network of ESP32 microcontrollers which communicate between themselves using ESP-NOW. One node interfaces with sensors for continuous monitoring of ammonia and temperature levels, while the other node runs a TinyML model trained on PC and deployed remotely using TensorFlow Lite library that predicts the environmental spikes before they occur, forming a closed-loop control system that triggers ventilation or turns on the heating element to mitigate the possible dangers of uncontrolled ammonia and temperature level. Furthermore, all predictive data and system analytics are aggregated via Wi-Fi to a centralized web dashboard and alert messages in case of critical conditions, providing farm administrators with a scalable, real-time dashboard for proactive agricultural management.

Keywords — *Ammonia Monitoring, Broiler, DHT22, Edge-AI, Environmental Control, ESP32, IoT, MQ-137, Real-Time Monitoring, TinyML, Ventilation Control*

TABLE OF CONTENTS

DECLARATION	i
CERTIFICATE OF APPROVAL	ii
COPYRIGHT	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS	x
1. INTRODUCTION	1
1.1. Background	2
1.2. Motivation	2
1.3. Problem Statement	3
1.4. Objectives	3
1.5. Scope and Limitations	4
1.6. Report Organization	5
2. LITERATURE REVIEW	6
2.1. Introduction to Poultry Farming Automation	6
2.2. Evolution of Architecture in IoT	6
2.3. Limitations of Traditional Control Loop: Navigating the Flaw	7
2.4. Machine Learning in Poultry Farming Automation	7
2.5. Identification of the Technical Gap	8
2.6. Availability of Datasets for Environmental Prediction	8
3. REQUIREMENT ANALYSIS	9
3.1. Project Requirement	9
3.2. System Requirement	11
3.3. Feasibility Analysis	14
4. SYSTEM ARCHITECTURE AND METHODOLOGY	16
4.1. System Architecture	16
4.2. Working Principle	20
4.3. Flowchart	21

5. IMPLEMENTATION DETAILS	25
5.1. Hardware Implementation	25
5.2. Software Implementation	31
5.3. Dataset Analysis	34
5.4. Model Architecture	38
5.5. Interfacing Technicalities and Protocols	39
6. RESULT AND ANALYSIS	41
6.1. Regression Model Performance	43
6.2. Classification Performance	49
6.3. Discussion	51
6.4. Result of Overall System	53
7. FUTURE ENHANCEMENTS	54
7.1. Additional Environmental Parameters	54
7.2. Diverse and Larger Dataset	54
7.3. Disease and Behavior Monitoring	54
7.4. Enhanced Sensor Technology	54
8. CONCLUSION	55
APPENDICES	56
A. APPENDICES	56
A.1. Project Timeline	56
A.2. Bill of Materials (BOM)	57
A.3. Circuit Diagram	58
A.4. PCB Diagram	60
A.5. Enclosure Design	62
A.6. Hardware Interconnection Table	63
A.7. Dataset	64
REFERENCES	66

LIST OF FIGURES

Figure 4-1.	System architecture	16
Figure 4-2.	ESP32 DevKit V1 slave node flowchart	21
Figure 4-3.	ESP32-S3 master node flowchart	23
Figure 4-4.	Closed-loop control system	24
Figure 5-1.	ESP32 DevKit V1 board	25
Figure 5-2.	ESP32-S3 microcontroller	25
Figure 5-3.	MQ-137 Ammonia Gas Sensor	28
Figure 5-4.	DHT22 Temperature and Humidity Sensor	29
Figure 6-1.	Distribution of temperature, humidity and ammonia concentration .	41
Figure 6-2.	Correlation matrix of environmental variables	42
Figure 6-3.	Actual vs predicted temperature on test set	43
Figure 6-4.	Actual vs predicted ammonia on test set	44
Figure 6-5.	Residual plot for temperature prediction	45
Figure 6-6.	Residual plot for ammonia prediction	46
Figure 6-7.	Prediction error distribution for temperature and ammonia	47
Figure 6-8.	Permutation importance for temperature and ammonia concentration	48
Figure 6-9.	ROC curve	49
Figure 6-10.	Confusion matrix	50
Figure 6-11.	Web dashboard showing live sensor readings, Edge-AI forecasts, and the console feed	53
Figure A-1.	Project Timeline Gantt Chart	56
Figure A-2.	Complete system schematic showing all subsystem interconnections	58
Figure A-3.	ESP32 configuration circuit used for sensor interfacing and data acquisition	58
Figure A-4.	Sensor interfacing circuit showing DHT22 and MQ-137 connections	59
Figure A-5.	RG1602A I2C LCD display interfacing circuit	59
Figure A-6.	Power supply circuit for the portable hardware system	59
Figure A-7.	Actuator control circuit showing fan and heater connections	60
Figure A-8.	PCB front layout design including relay driver and LCD display placement	60
Figure A-9.	PCB back layout design for the smart poultry monitoring system .	61
Figure A-10.	3D rendered view of the final assembled PCB with LCD display . .	61
Figure A-11.	3D rendered design of the protective hardware enclosure box showing the base panel and ventilated front grille	62

LIST OF TABLES

Table 3-1. Hardware components used in the system	11
Table 3-2. Software components used in the system	12
Table 4-1. Ammonia concentration risk levels	18
Table 4-2. Suitable temperature by week for broilers	19
Table 5-1. Dataset split distribution	34
Table 5-2. Raw sensor variables in the dataset	35
Table 5-3. Model input features	35
Table 5-4. Model target variables	36
Table 6-1. Overall regression performance on the test set	47
Table 6-2. Spike classification performance metrics (test set)	50
Table A-1. Project Budget	57
Table A-2. System Hardware Interconnection Map	63

LIST OF ABBREVIATIONS

ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
AUC-ROC	Area Under the ROC Curve
BOM	Bill of Materials
DHT22	Digital Humidity and Temperature Sensor
ESP-NOW	Espressif Protocol for Low-Power Device Communication
ESP32	Espressif Systems Processor 32
GPIO	General Purpose Input/Output
GPU	Graphical Processing Unit
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
LCD	Liquid Crystal Display
MAE	Mean Absolute Error
ML	Machine Learning
MLP	Multilayer Perceptron
MQ-137	Gas Sensor for Ammonia Detection
MSE	Mean Squared Error
NIH	National Institutes of Health
NPR	Nepalese Rupee
NTC	Negative Temperature Coefficient
PC	Personal Computer
PMC	PubMed Central
ppm	Parts Per Million
PTC	Positive Temperature Coefficient
PTQ	Post-Training Quantization
PWM	Pulse Width Modulation
RH	Relative Humidity
ROC	Receiver Operating Characteristics
SPI	Serial Peripheral Interface
TFLite	TensorFlow Lite
TinyML	Tiny Machine Learning
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
Wi-Fi	Wireless Fidelity

1. INTRODUCTION

The rapid growth of Internet of Things (IoT) has shaped the agricultural system particularly by monitoring and managing through use of IoT. In particular, poultry farming has benefited significantly from these advancements like introduction of smart poultry solutions that improves efficiency of production. By combining IoT sensors with data-driven techniques, it is now possible to observe and maintain the environment inside poultry houses and respond manually to changes that affect bird's health and productivity [1], [2].

Broilers are raised in controlled environments by maintaining stable conditions for optimal growth in modern broiler farming. Parameters such as temperature, humidity and ammonia concentration directly affect the poultry health and yield of farms. Among these, ammonia is an especially important parameter because even a moderate increase can lead to respiratory problems, degradation of the immune system and lower productivity. Also temperature deviations during first week can significantly increase mortality rate, so maintaining a stable microclimate is crucial for healthy development of broiler [3], [4], [5], [6], [7].

However, in many poultry farms, environmental monitoring through IoT is applied but it is partially automatic or relies on threshold based systems. These approaches are significant upgrades from fully manual farms but they fail to provide deeper insights on harmful conditions and time based prediction for these conditions. As a result farmers only react after the environment has already become critical and has already affected the health of birds. These limitations highlights need for improvement in these system with smarter and responsive monitoring approach that can not only observe conditions, analyze and react supporting early decision making.

1.1. Background

Poultry farming is an important part of Nepal's agricultural sector. In broiler production, maintaining a suitable environment inside the poultry house is important for healthy growth and good production performance. The brooding period, particularly the first two weeks after hatching, is one of the most sensitive stages, as young chicks are highly affected by changes in their surroundings. Temperature, humidity, and ammonia concentration are among the important environmental factors that affect their health and growth. Very high or low temperatures can cause stress and slow down growth, high humidity can result in poor litter conditions, and the accumulation of ammonia can cause respiratory problems and negatively affect overall productivity [8].

In Nepal, many poultry farms still use traditional open-house systems. These houses are strongly affected by outdoor weather conditions, making it difficult for farmers to maintain a stable environment inside. In many cases, farmers depend on their own experience and regular observation to check conditions and decide when management actions are needed. Although this approach can be useful, it does not provide continuous information about the actual environmental conditions, and changes in these parameters may go unnoticed until they begin to affect the chicks.

The development of sensors and wireless communication technologies has provided new possibilities for monitoring poultry house environments. IoT-based systems can collect environmental data continuously, allowing farmers to monitor conditions in real time. However, many existing systems are mainly designed to display sensor readings or provide simple alerts, offering limited analysis of the data and little support for predicting future environmental conditions [9]. Therefore, there is a need for a more effective monitoring approach capable of collecting and analyzing continuous environmental information inside poultry houses, helping farmers identify unfavorable conditions earlier and take appropriate action during the critical brooding stage of broiler production.

1.2. Motivation

The idea for this project came from the discussions about the bird flu outbreaks that occurred in Nepal in late May. This made us realize that poultry farming can be affected by health risks that may lead to significant losses for farmers. Although directly detecting or preventing diseases such as bird flu was beyond the scope of our minor project, we wanted to focus on an area where we could make a practical contribution. We therefore decided to improve the existing poultry farming particularly on broiler during the brooding stage by focusing on environmental conditions that can affect poultry health. Environmental parameters such as temperature, humidity and ammonia are important

for maintaining suitable conditions inside a poultry house. Instead of only monitoring, we aimed to use IoT sensors to collect real-time data and machine learning analyze the data. This allows identification of possible changes in environmental conditions of the broiler in brooding stages.

The main motivation of this project is to develop a system that can provide farmers with earlier awareness of the unfavorable conditions and take appropriate actions. While the system cannot directly detect or prevent diseases such as bird flu, maintaining better environmental conditions can help to reduce the mortality rate, risk and support better farming for broilers.

1.3. Problem Statement

Poultry farming in Nepal commonly uses an open-house system. Such systems are strongly affected by outdoor weather and day-to-day management practices. Maintaining suitable conditions of farm is important during the first two weeks of broiler growth, as young chicks are sensitive to changes in temperature and air quality. Continuous monitoring and maintenance for environmental conditions can be difficult for farmers, particularly when they mainly depend on manual observation and experience.

Temperature and ammonia concentration are two major environmental concerns during brooding period. When the temperature and ammonia level rises beyond the safe level, chicks can experience stress, reduced growth, and higher risk of mortality. Higher ammonia levels can affect the respiratory health of birds and reduce overall productivity. Changes in humidity can make it difficult to maintain suitable litter contribution and may contribute to increased ammonia levels [10].

Although some farms use basic methods to monitor environmental conditions, continuous monitoring of temperature, humidity, and ammonia is still limited. Farmers may not always be aware of unfavorable changes as they occur, delaying the necessary response. This is particularly concerning during the brooding period. Therefore, there is a need for a system able to monitor these environmental parameters continuously, providing timely information.

1.4. Objectives

The main objectives of this project are as follows:

- To design and develop an IoT-based Edge-AI system for real-time monitoring, prediction, and control of environmental conditions in poultry houses, with a focus on the brooding stage of broiler production.

1.5. Scope and Limitations

1.5.1. Scope

The current project aims to develop an IoT and machine learning based system for monitoring and managing environmental conditions. The system focuses on real-time monitoring of temperature, humidity and ammonia concentration using sensors connected to an ESP32. The collected environmental data is processed and used by learning model to predict environmental changes. It notifies early warnings of potentially unsafe conditions. The system also includes automated ventilation control and web-based dashboard for real-time visualization. The application domain includes poultry-house environmental monitoring, early detection of unfavorable conditions, predictive environmental management and ventilation control. The project is mainly focused on developing and evaluating a prototype under the condition represented by collected dataset. The current scope does not include disease diagnosis, full scale commercial farm integration and poultry behavior monitoring.

1.5.2. Limitations

The scope of this project is limited to environmental monitoring, prediction, visualization, alerting, and ventilation control based on temperature, humidity, and ammonia levels. The performance of the machine-learning model is dependent on the quality, size, and range of the available dataset. Additional environmental parameters and advanced biological health monitoring, such as sound or image-based disease along with behavior detection are not implemented in the current system.

1.6. Report Organization

This report is organized into eight chapters, with each chapter covering a specific aspect of the developed smart poultry farming system. The **Introduction** chapter presents the background of the project, identifies the problem and describes the objective and scope of the developed system. The **Literature Review** chapter shares reflections on previously published projects of a similar kind. The **Requirement Analysis** chapter is divided into three parts: Project Requirement, System Requirement and Feasibility Analysis. The **System Architecture and Methodology** chapter illustrates the overall system architecture, working picture and dataflow used to develop the system. The **Implementation Details** chapter describes the practical implementation of the hardware and software components including the sensor integration and its interfacing along with the protocols used for communication. The **Result and Analysis** chapter presents the experimental results obtained from the developed system and evaluates the performance of the prediction model with the system operation. The **Future Enhancements** chapter discusses the possible improvements and extensions that can be incorporated into the system in future work. Finally, the **Conclusion** chapter summarizes the overall work, major achievements and significance of the developed system. Additional technical information including the circuit diagram, PCB diagram, module specification and other supporting materials is provided in the **Appendices** for reference.

2. LITERATURE REVIEW

2.1. Introduction to Poultry Farming Automation

The Internet of Things (IoT) has been integrated into poultry farming automation through the use of multiple sensors and actuators to improve environmental monitoring and farm management [3], [11]. This represents a major transition from traditional agricultural approaches toward automated and data-driven farming systems. The biological impact of microclimates plays a vital role in poultry health and mortality rates. Environmental parameters such as temperature and NH_3 (ammonia gas) concentration are among the major factors that directly affect poultry growth and production performance [5], [12].

Chicks in the earliest stage of life (1-3 weeks) are extremely sensitive to microclimate variables, especially temperature and ammonia concentration. The mortality rate during this period is one of the most significant economic challenges in poultry for broilers.

Unlike adult birds, a day-old chick cannot adapt to temperature making them entirely dependent on surrounding conditions to be warm [13]. Due to this poor environmental management that allows cold stress to occur and improper handling increases first week mortality up to three times [14]. Alongside the temperature, the ammonia in the air plays another role in mortality of chicks, if the ammonia concentration reaches 25ppm, the damage to developing respiratory tracts of chicks is caused and increases susceptibility to disease and reduction in feed intake [14], [15], [16].

Therefore, continuous monitoring of these parameters is essential for maintaining a suitable poultry house environment.

2.2. Evolution of Architecture in IoT

The microcontroller acts as the central processing unit that processes sensor data and controls environmental actuators [3], [5]. These environmental control mechanisms commonly include ventilation systems and temperature regulation systems that operate based on monitored environmental conditions [3], [5].

Sensors such as DHT11/DHT22 for temperature and humidity monitoring and MQ-137 for ammonia gas detection are widely used in low-cost poultry automation systems [12], [17]. However, DHT22 has $\pm 0.5^\circ\text{C}$ of accuracy vs DHT11 has accuracy of $\pm 2^\circ\text{C}$. Also, for the ammonia sensor MQ-137 is reported to have accuracy of $R^2 = 0.64$ in real-life conditions [2]. These sensors provide sufficient environmental information while maintaining low implementation costs, making them suitable for prototype and small-scale smart poultry farming applications.

2.3. Limitations of Traditional Control Loop: Navigating the Flaw

Many existing poultry monitoring systems rely on threshold-based control mechanisms [3], [11]. These systems use current sensor readings to trigger environmental control actions. For example, when the temperature exceeds a predefined threshold, the ventilation fan is activated.

Although effective for basic automation, this approach remains reactive. The control mechanism responds only after unfavorable environmental conditions have already occurred. The temperature swing outside the safe range increases mortality risk in early aged chicks because they face severe thermal stress and metabolic exhaustion [14]. Consequently, poultry may be exposed to stressful conditions due to which body weight declines roughly by 6% and 9% at 50 ppm and 75 ppm respectively, compared to unexposed birds, also with mortality rising to 13.9% at 75ppm and 5.8% in unexposed environment (0 ppm) [18]. This limitation becomes particularly important in the case of ammonia accumulation, where delayed intervention may negatively affect poultry health and productivity.

2.4. Machine Learning in Poultry Farming Automation

Recent advancements in artificial intelligence and machine learning have enabled more intelligent approaches to poultry farm monitoring and management [4]. Unlike traditional threshold-based systems that rely solely on current sensor measurements, machine learning models can analyze historical and real-time environmental data to identify patterns and trends.

Instead of relying only on the current environmental state, the Multilayer Perceptron model can utilize a sequence of observations collected over a continuous time window. By analyzing variations in ammonia concentration, temperature, and humidity over time, the model can identify environmental trends and provide early warnings of potentially unsafe conditions before critical thresholds are reached [4]. Since the multilayer perceptron (MLP) demonstrated superior accuracy in predicting the spikes in temperature and ammonia concentrations in air compared to ML models like Decision Tree, Naive Bayes, which justifies the selection of MLP for the minor project [15].

Such predictive capabilities allow environmental control mechanisms to respond proactively rather than reactively, helping maintain a more stable poultry house environment and improving overall poultry welfare.

2.5. Identification of the Technical Gap

Existing literature demonstrates significant progress in IoT-based poultry monitoring systems and environmental automation [3], [11], [5]. However, most existing systems continue to focus primarily on reactive control strategies based on predefined threshold values.

While recent studies have highlighted the potential of artificial intelligence in poultry farming [4], limited research has explored the integration of low-cost IoT hardware with neural network-based models for short-term prediction of environmental conditions in poultry houses. Furthermore, the use of a continuous environmental observation window to predict potential spikes in ammonia concentration, temperature, and humidity before threshold violations has not been extensively investigated. Also, another gap that is identified is no existing low cost hardware implementation has worked on age based weekly threshold monitoring profiles despite broilers requiring specific temperature requirements changing significantly from week to week [19], [13], [17].

Therefore, this project aims to address this gap by integrating low-cost environmental sensors with a neural network model capable of analyzing environmental trends and providing early warnings of potentially unsafe conditions before they occur.

2.6. Availability of Datasets for Environmental Prediction

Public datasets are widely available for parameters like temperature, ammonia (NH₃), humidity and more from IEEE DataPort [20] and (PMC), a repository maintained by the National Institutes of Health (NIH) [21] for analysis of correlation and to establish a realistic environment for broilers. Also, the closely related datasets involve MDPI [22] datasets which involve the parameters required for us.

3. REQUIREMENT ANALYSIS

3.1. Project Requirement

The project requirement are categorized into functional and non-functional requirement to ensure that system meets its intended purpose objective. These requirements describes the core functionality and quality attributes along with operational characteristics. This ensured the system operates reliably and as expected during implementation.

3.1.1. Functional Requirements

The successful implementation of an Edge-AI smart poultry system requires a clear definition of its main function. These functional requirements describe how the system collects environmental data and process the sensor readings. The system describes its response to unsafe situation and predict future conditions. It provides a basis for developing and evaluating the overall performance of the system.

- The system shall continuously monitor the important environmental parameters inside the poultry house including temperature, humidity, and ammonia concentration.
- The system shall transmit the collected sensor data from sensing node to the central processing node using a low-power wireless communication protocol.
- The system shall utilize a trained Multi-Layer Perceptron model on the central node to predict future environmental conditions using previous sensor measurements.
- The system shall classify the environmental conditions into predefined risk level, allowing system to identify the potentially hazardous conditions before they reach critical levels.
- The system shall automatically activate the appropriate control devices such as ventilation fan or heating element when the environmental conditions exceed the defined safely limits.
- The system shall provide a web-based dashboard that displays both real-time sensor measurements and predicted environmental parameters for continuous monitoring.
- The system shall notify the poultry farm operator when hazardous conditions are predicted.
- The system shall store the collected sensor measurements and prediction results for further analysis, evaluation, and assessment of the overall system performance.

3.1.2. Non-Functional Requirements

Non-functional requirements are necessary for governing the operational success in a poultry house with factors such as reliability, response time, scalability, maintainability, and power efficiency. These non-functional requirements tell how well the system runs. Defining these requirements is essential such that the final deployment is stable, sustainable, and resilient in the farm environment.

- The system shall process sensor reading and predict the future value with low latency for real-time environmental monitoring and control.
- The system shall maintain data packet delivery rate of at least 95% between sensing and processing nodes under normal conditions.
- The system shall support scalability and be able to add additional sensor nodes or control devices without big modification to the existing architecture.
- The system shall be designed using modular architecture for easy maintenance, debugging, and future enhancements.
- The system shall provide a feature a simple web-based dashboard that can be used without requiring advanced technical knowledge.
- The system shall employ low-power microcontrollers and low-power wireless protocols to minimize the overall energy during continuous operation.
- The developed ML model shall be portable and compatible with ESP32 based embedded platforms.
- The system shall ensure reliable communication between the sensor and master nodes, preventing unauthorized modification of sensor readings.

3.2. System Requirement

3.2.1. Hardware Components

Table 3-1 lists the hardware components used in the system, where each one is used, and its purpose.

Table 3-1. Hardware components used in the system

Component	Where it is used	Purpose
ESP32 DevKit V1	Sensor/control board	Collects sensor readings from the DHT22 and MQ-137 sensors and sends the data to the master ESP32-S3.
ESP32-S3	Master/controller board	Acts as the main controller. It communicates with the Django backend over Wi-Fi, processes the received model and control information, and communicates with the other ESP32 board using ESP-NOW.
DHT22	Environmental sensing section	Measures temperature and relative humidity, providing the data needed for monitoring and prediction.
MQ-137	Gas sensing section	Measures ammonia (NH ₃) concentration, which is important for detecting potentially hazardous gas levels.
Fan	Ventilation/control section	Provides ventilation inside the poultry house and is controlled according to the system's predicted environmental conditions.
PTC Heater	Heating/control section	Provides heating when the temperature drops below the lower threshold.
Relay Module	Heater control circuit	Acts as an electrically controlled switch for the PTC heater, letting the ESP32's digital GPIO signal control the heater.
LCD	Local monitoring/interface	Displays current temperature and ammonia readings.
LM2596 DC-DC Buck Converter	Power supply section	Steps down the input DC voltage to a level suitable for the electronic components and helps supply the system with appropriate power.
Resistors	Sensor and control circuits	Used for electrical interfacing and to protect the electronic components as required.

Table 3-1 – continued from previous page

Component	Where it is used	Purpose
Power Supply	Overall hardware system	Provides the electrical power required by the ESP32 boards, sensors, display, control circuits, fan, and heater.
PCB	Hardware assembly	Provides the physical platform for mounting and electrically connecting the components into the final system.

3.2.2. Software Components

Table 3-2 lists the software and technologies used to develop and deploy the system, along with their purpose.

Table 3-2. Software components used in the system

Software Technology	/ Where it is used	Purpose / Why it is required
Python	ML development and data processing	Main programming environment for processing the sensor dataset and developing the machine learning model.
TensorFlow Keras	/ MLP development	Used to build, train, and evaluate the multi-task MLP model for regression and classification.
NumPy	ML/data processing	Used for numerical operations and array manipulation.
Pandas	Dataset processing	Used to load, organize, clean, and process the sensor dataset before training the model.
Scikit-learn	ML evaluation/preprocessing	Used for machine learning preprocessing and model evaluation, including performance metrics.
Matplotlib	Results visualization	Used to generate graphs for analysing model performance and sensor/model results.
PlatformIO	ESP32 development	Used to write, compile, upload, and manage the firmware running on the ESP32 DevKit V1 and ESP32-S3.

Table 3-2 – continued from previous page

Software Technology	/ Where it is used	Purpose / Why it is required
Django	Backend server	Provides the web backend that communicates with the ESP32-S3, receives sensor and prediction data, and handles control information.
HTML, CSS	Dashboard interface	Define the structure and content, and control the visual appearance and layout, of the dashboard.
JavaScript	Dashboard functionality	Provides interactive dashboard functionality and lets the interface communicate with the backend for monitoring and control.
SQLite	Database	Stores the system data handled by the Django application, providing persistent local storage.
TensorFlow Lite (TFLite)	ESP32 deployment	Used to convert and deploy the trained MLP model in a compact format suitable for embedded inference on the ESP32.
Wi-Fi / HTTP / JSON	ESP32-S3 ↔ Django communication	Wi-Fi provides the network connection, while HTTP and JSON are used to exchange sensor, prediction, and control data between the master ESP32 and the Django backend.
UDP	Server discovery	Used by the ESP32-S3 to discover the Django server's IP address.
ESP-NOW	ESP32-to-ESP32 communication	Provides communication between the ESP32-S3 master and the ESP32 DevKit V1 sensor node.

3.3. Feasibility Analysis

3.3.1. Technical Feasibility

The developed system utilizes hardware components including ESP32 DevKit V1, ESP32-S3, DHT22 and MQ-137 sensors which are capable of monitoring environmental parameters in poultry houses. The ESP32 DevKit V1 is used as a data acquisition node while ESP32-S3 is used for running inference locally. The ML model is trained using TensorFlow and deployed using TensorFlow Lite. Communication between the two nodes is done through ESP-NOW which is reliable and low-latency wireless data transmission protocols for ESP32 devices. The machine learning model is developed using Python with TensorFlow and Keras Library. Other libraries such as Pandas and NumPy are used for data processing and calculations while Scikit-learn is used for model evaluation. Publicly available poultry environment datasets are used for model development and are further supplied with real-time sensor readings for testing and validation. Since all the hardware, software and datasets are readily available and compatible with each other making the system technically feasible for implementation.

3.3.2. Economic Feasibility

The developed system is economically feasible because it uses low-cost embedded hardware and open-source free software tools. The major hardware components include the ESP32 microcontrollers, DHT22 sensor, MQ-137 sensor, relay module, PWM fan and heating element which are easily available in the market at relatively low cost. The overall implementation cost of the developed system remains less than that of commercially available poultry monitoring systems so it is affordable for small and medium-scale poultry farms. Software tools and development environments used are entirely free and open-sourced like PlatformIO, Python-based libraries and frameworks. The absence of software purchases and licensing costs reduced the overall development expenditure. In addition, the modular design of the system allows other hardware components to be added or upgraded without extra effort of redesigning the entire system minimizing future maintenance costs.

3.3.3. Operational Feasibility

The system is designed for continuous monitoring and automatic control of poultry environment with less human intervention. Necessary environmental parameters like temperature, humidity and ammonia are acquired through the sensors by ESP32 DevKit V1 and transmitted to the ESP32-S3 for predicting future environmental conditions. Based on the prediction results, the system controls the necessary actuators and also displays real-time information on the web-dashboard. The user interface is designed to be simple allowing users with no technical knowledge to use the system effectively with ease. It allows them to monitor the environmental conditions and receive alerts during hazardous conditions. The system uses commonly available hardware, wireless communication and edge computation for decision-making which makes it deployable in small and medium-scale poultry farms with minimal efforts on operation.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. System Architecture

The overall system architecture, from environmental sensing through edge prediction to actuator control, is illustrated in Figure 4-1.

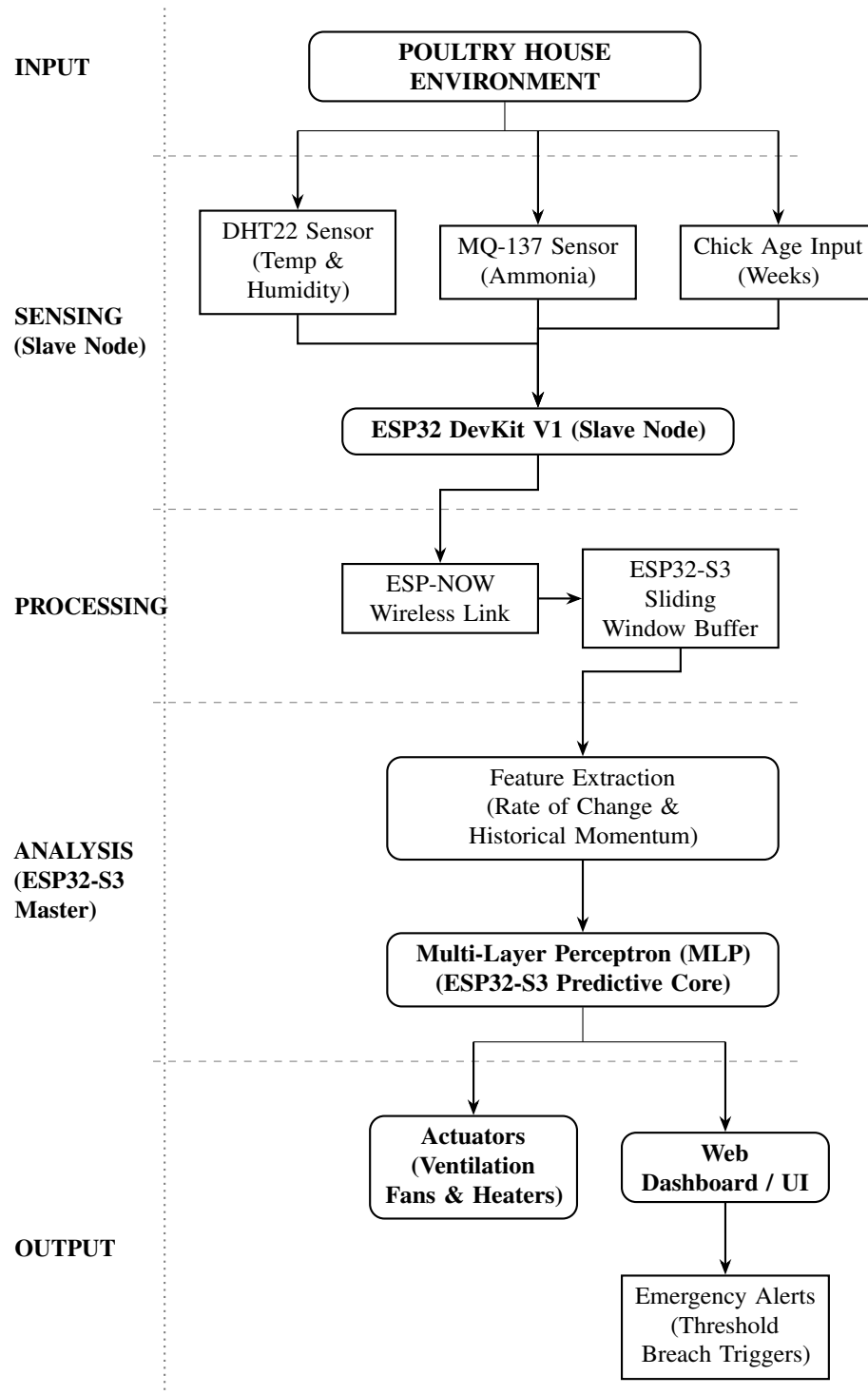


Figure 4-1. System architecture

4.1.1. Input Layer

The Temperature and humidity are measured continuously using the DHT22 sensor. Ammonia concentration is measured using the MQ-137 sensor. These sensors are part of the first Data Acquisition node which collects the data, and processes the data to be sent to the processing layer for further decision-making.

The input layer consists of four features, each of which is derived from sensors and input.

- **Temperature (current reading):** This will be the most recent temperature value that is recorded by DHT22 sensor, and it is expressed in °C, this represents the instantaneous temperature of our poultry farm. This will be compared to our baseline temperature based on different weeks of chicks, the temperature is higher for lower aged broilers and this decreases as the age increases.
- **Humidity (current reading):** This is also the most recent relative humidity recorded from the DHT22 sensor itself and is expressed as % and it is an important parameter because it is directly linked to the accelerated production of ammonia in poultry farms [10].
- **NH3 (current reading):** This is the most recent ammonia concentration in the farm which is recorded by the MQ-137 sensor and is expressed in ppm. This is the primary parameter for our objective in predicting the NH3 reading, as the NH3 exceeds the threshold 15 ppm it will be the danger zone, so our objective is to prevent the spike and keep the concentration below threshold.
- **Chick Age Week (1–4):** This is categorized from 1–4 weeks, this is used because parameters like ammonia, temperature and humidity changes based on the age of chicks. This input allows the model to learn age dependent patterns rather than specific threshold patterns.

4.1.2. Processing Layer

Sliding-Window Multi-Layer Perceptron

The system is designed and deployed at the edge using a Multi-Layer Perceptron. The MLP was selected because it can fit easily in the ESP32, is compatible with TensorFlow Lite, and is the most efficient model based on the findings of Barbosa et al. [15].

It is a time-related problem to predict future values. The raw readings will not be sufficient for model inputs, so to fix this, a sliding-window approach is adopted. At each inference step, the model receives the most recent sensor readings for the sliding window. The model is trained to predict future values and provide decisions before a critical threshold is reached.

Model Training & Deployment Workflow

The MLP model is trained using data from the poultry environment. The model training is done on a PC using TensorFlow. After training on the PC, the model is converted into TensorFlow Lite format using the TFLiteConverter API with Post-Training Quantization (PTQ). Quantization decreases the model size and inference time without substantial degradation in accuracy. The resulting .tflite model is flashed into the ESP32 firmware and executed using the TFLite Micro inference engine.

Ammonia Concentration Risk

Ammonia concentration thresholds were derived from established research papers with target setpoints, which are listed in the table below [15]. Criteria used for creating the target attribute risk of ammonia concentration for broilers.

Ammonia Concentration	Risk Level
No risk	0–1 ppm
Low risk	2–9 ppm
Moderate risk	10–14 ppm
High risk	15–20 ppm
Very high risk	>21 ppm

Table 4-1. Ammonia concentration risk levels [15]

Suitable Temperature by Weeks

Age-based temperature thresholds were derived from established research papers with target setpoints, which are listed in the table below [17]. The temperature values are adjusted based on industrial broiler standards [6], [7].

Week	Temperature Range (°C)
Week 1	32–35
Week 2	29–32
Week 3	27–29
Week 4	24–27
Week 4+	20–22

Table 4-2. Suitable temperature by week for broilers [17], [6], [7]

4.1.3. Output Layer

The output layer completes the control loop by translating the MLP’s predictions into physical adjustments and remote updates.

- **Preemptive Climate Control:** Instead of waiting for a threshold to be breached, a preemptive Pulse-Width Modulation (PWM) fan uses the MLP’s output as its input. Based on the criticality of the ammonia spike, the system decides the fan’s speed to neutralize the hazard before it impacts the chick’s health.
- **Web Dashboard Syncing:** The real-time sensor readings and corresponding model predictions are synced via Wi-Fi to a web-based monitoring dashboard. This provides clear visual trends of the current and predicted values of ammonia and temperature.
- **Emergency Alerting:** If the MLP flags a critical hazard, the master node sends an automated alert, ensuring the farmer can step in manually to prevent the hazard.

4.2. Working Principle

The developed system uses different sensors along with Tiny Machine Learning (TinyML) to monitor and analyze and control the environment of the farm in real time. The system uses two ESP32 modules, where one handles the input section that comes from sensors and the other handles processing, predicting and decision-making.

First the ESP32 DevKit V1 collects data from DHT22 and MQ-137 sensors where it measures the temperature, humidity and ammonia concentration respectively. The collected data is sent to the master node (ESP32-S3) which processes the input and predicts ammonia and temperature after 10 minutes. ESP32 DevKit V1 and ESP32-S3 uses ESP-NOW to communicate wirelessly without the internet.

The master node during processing analyzes the sensor data for understanding real-time conditions of poultry. A TinyML model is then used to predict future ammonia concentration and temperature of poultry. If the conditions are normal, it keeps monitoring endlessly. If any spikes or abnormalities are detected, based on the abnormality either ventilation system is activated or heating element is activated or both. The system also notifies the farmer through the web dashboard when these abnormalities are detected.

Overall, the system monitors and maintains the environment continuously using sensors, wireless communication, machine learning model and actuators to maintain a safe environment for broilers.

4.3. Flowchart

The firmware is divided into two physical nodes: the Data Acquisition & Actuation Node (Slave) and the Edge-AI & Predictive Control Node (Master). The Slave handles sensor readings and actuator control, while the Master performs the ML processing and prediction. The overall process is shown in the system flowchart.

4.3.1. Flowchart 1: ESP32 DevKit V1 Slave Node

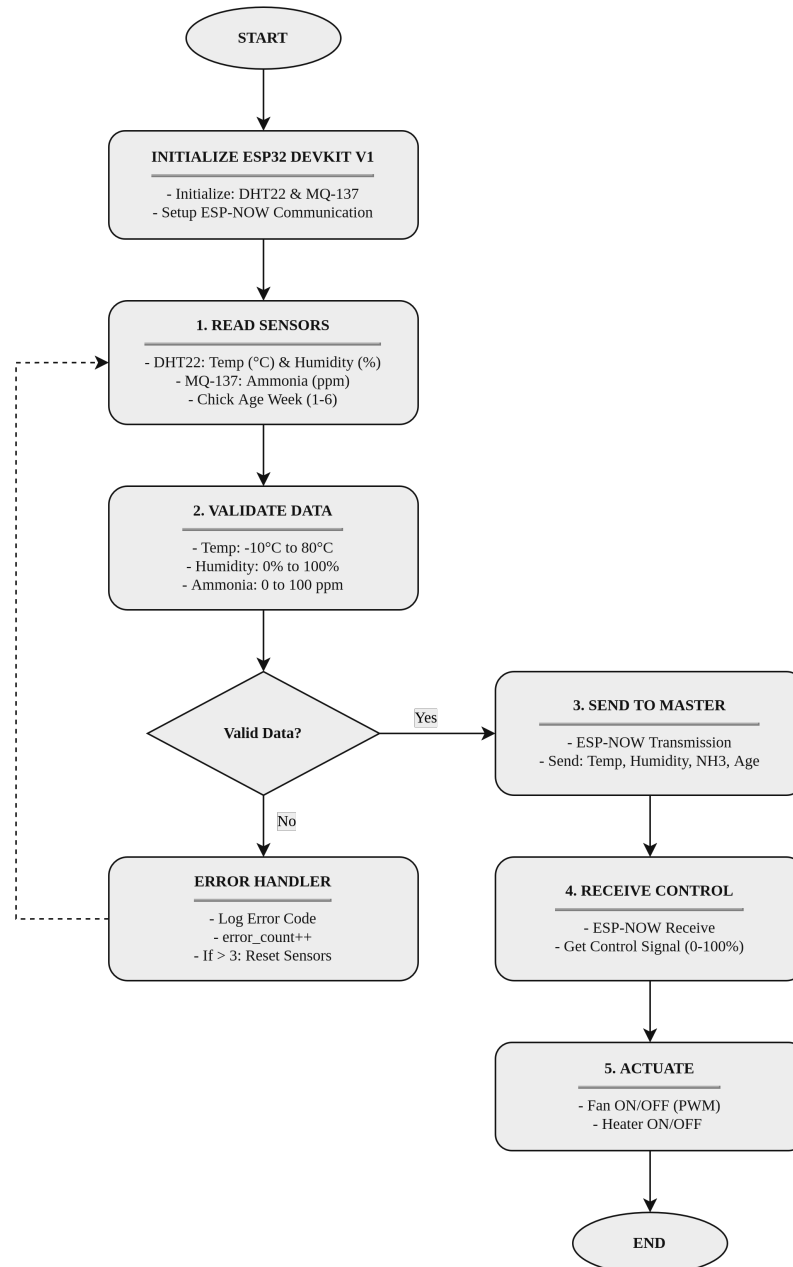


Figure 4-2. ESP32 DevKit V1 slave node flowchart

1. The board powers on, connects with DHT22 and MQ-137 sensors, and links to the main board using ESP-NOW.
2. It constantly reads the temperature, humidity, and ammonia levels inside the poultry house.
3. It needs input of chicks' week age to understand its target temperature.
4. Before sending to the Master node, it checks if it's valid or not, with certain criteria (Temp -10 to 80°C, ammonia 0 to 100 ppm).
5. If the sensor does not pass the criteria, it gives an error, and after 3 times, the board restarts the sensors automatically to fix the issue.
6. After sending, it waits for data from the Master node for the actuator control.

4.3.2. Flowchart 2: ESP32-S3 Master Node

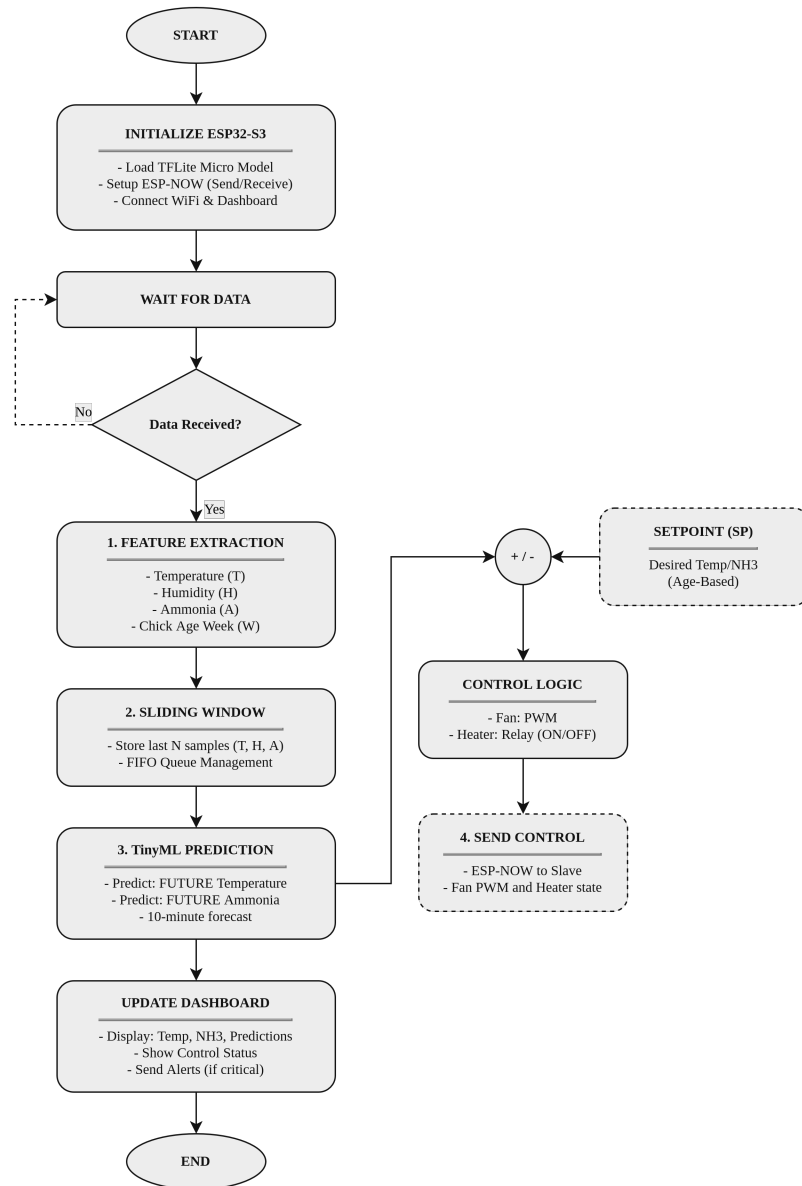


Figure 4-3. ESP32-S3 master node flowchart

1. The board powers on, connects with Wi-Fi, and loads the TFLite model into its memory.
2. It waits for the slave node to send its recent sensor reading.
3. It saves a few past readings in a queue. This allows it to see if the temperature and ammonia levels are rising or dropping.
4. It runs the model to calculate the temperature and ammonia concentration 10 minutes ahead.
5. It uploads live data and the 10-minute prediction to the web dashboard for monitoring.
6. It compares the predicted values with the setpoint value input according to chicks' week age.

The calculated actuator value is sent back to the ESP32 DevKit V1 through ESP-NOW to regulate fans and heaters.

4.3.3. Closed-Loop Control System

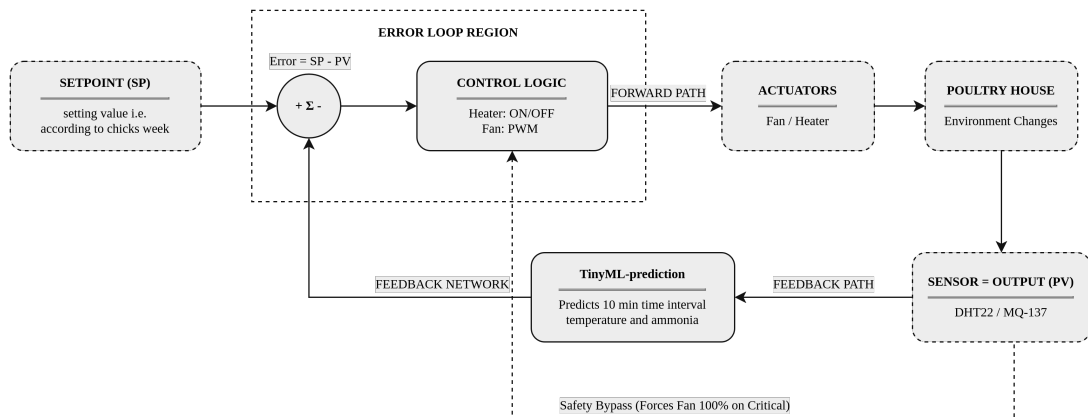


Figure 4-4. Closed-loop control system

1. The system selects set point value, the ideal values according to the chicks' age.
2. It constantly compares the setpoint value with the predicted value to get the error and how much correction it will take to make error 0.
3. It changes the speed of the fan and turns on/off the heating system based on the error and to fix the error.
4. If the environment gets spike in temperature or ammonia, then it bypasses the prediction and goes directly to the actuator.

5. IMPLEMENTATION DETAILS

5.1. Hardware Implementation

5.1.1. ESP32 Microcontroller

ESP32 is a low power microcontroller developed by Espressif Systems for embedded systems and Internet of Things. It is a 32-bit dual core microcontroller with integrated 2.4GHz Wi-Fi and Bluetooth. It has multiple GPIO Pins and peripherals such as ADC, SPI, I²C, UART and PWM to interface with sensors and actuators [23].

ESP32-S3 is a 32-bit microcontroller SoC from Espressif Systems for embedded systems, IoT and Edge-AI applications. It has dual core Xtensa LX7 processor, integrated 2.4 GHz Wi-Fi and Bluetooth Low Energy (BLE). It has 45 programmable GPIO pins and supports various peripheral interfaces. It supports high-speed SPI flash and a PSRAM with configurable data and instruction cache. It has additional support for vector instructions providing enhanced computations for neural network and signal processing [24].

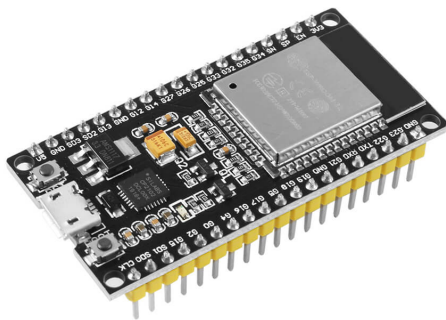


Figure 5-1. ESP32 DevKit V1 board [25]



Figure 5-2. ESP32-S3 microcontroller [26]

5.1.2. MQ-137 Ammonia Gas Sensor

The MQ-137 is a semiconductor gas sensor designed for detection of ammonia (NH₃). The sensor is encapsulated with a Bakelite base and a metal cap holding the sensing element and the heater assembly. Tin Dioxide (SnO₂) is the sensitive material used in the sensor for main detection, whose electrical conductivity increases with an increase in ammonia concentration. According to the manufacturer's specifications, the MQ-137 has a detection range of 5–500 ppm and operates under 5.0 ± 0.1 V. The sensor is required

to be heated for more than 48 hours before use to achieve stable operating characteristics under the specified test conditions [27].

Working Mechanism

The MQ-137 operates based on the principle of metal oxide semiconductor (MOS) gas sensing. It consists of a tin dioxide (SnO_2) sensing layer and a built-in Positive Temperature Coefficient (PTC) heating element that maintains the sensing material at an operating temperature of approximately 200–300°C. In clean air, oxygen molecules adsorb onto the SnO_2 surface and capture free electrons, increasing the sensor's resistance. When ammonia gas is present, it reacts with the adsorbed oxygen ions, releasing the trapped electrons and decreasing the sensor's resistance. This resistance change produces a corresponding variation in the analog output voltage.

The analog signal is sampled by the ESP32's 12-bit Analog-to-Digital Converter (ADC), converted into the sensor resistance (R_s), and compared with the calibrated reference resistance (R_0). The ammonia concentration is then estimated using the sensor's calibration equation:

$$V_{out} = \left(\frac{ADC}{4095} \right) \times V_{ref} \quad (5-1)$$

where ADC is the digital reading from the 12-bit ADC and V_{ref} is the reference voltage (typically 3.3 V).

The sensor resistance (R_s) is then calculated using the voltage divider relationship:

$$R_s = R_L \left(\frac{V_{out} - V_{ref}}{V_{out}} \right) \quad (5-2)$$

where R_L is the load resistance connected to the sensor.

To determine the ammonia concentration, the resistance ratio R_s/R_0 is first computed, where R_0 represents the sensor resistance under a known calibration concentration. The ammonia concentration is then estimated using the sensor's logarithmic sensitivity characteristic:

$$PPM = A \left(\frac{R_s}{R_0} \right)^B \quad (5-3)$$

where A and B are experimentally determined calibration constants obtained from the MQ-137 sensitivity curve provided by the manufacturer. The calculated ammonia

concentration, expressed in parts per million (ppm), is subsequently transmitted to the prediction model running on the ESP32-S3 for environmental forecasting and control.

Sensor Calibration

The MQ-137 utilizes a tin dioxide (SnO_2) semiconductor layer whose electrical surface resistance (R_s) varies inversely with the concentration of target reducing gases, specifically ammonia (NH_3).

Sensor Resistance Derivation (R_s)

The microcontroller cannot measure resistance directly but can only measure voltage. The sensor is therefore wired with a standard load resistor to create a voltage divider. By reading the analog voltage output (V_{out}), the system can determine the sensor's actual, real-time resistance using the formula below:

$$V_{out} = V_c \frac{R_L}{R_s + R_L} \quad (5-4)$$

Rearranging to solve for the dynamic sensor resistance R_s :

$$R_s = R_L \left(\frac{V_c - V_{out}}{V_{out}} \right) \quad (5-5)$$

where R_L is the physical load resistance on the board, V_c is the supply voltage applied to the circuit, and V_{out} is the digitized voltage obtained by scaling the raw Analog-to-Digital Converter (ADC) counts (A_{avg}):

$$V_{out} = \left(\frac{\sum_{i=1}^N A_i}{N \cdot \text{ADC}_{max}} \right) V_{ref} \quad (5-6)$$

Baseline Calibration (R_0 Determination)

The sensor characteristic curve relies on the resistance ratio R_s/R_0 , where R_0 represents the sensor resistance in clean air at standard atmospheric conditions (20°C, 65% RH). To calculate R_0 , the sensor must be preheated for 48 hours to achieve stable operating condition. When exposed to clean baseline air, the measured sensor resistance is denoted $R_{s,air}$. According to the manufacturer's datasheet, the ratio of $R_{s,air}$ relative to R_0 in clean air is a constant scaling factor K_{air} (typically $K_{air} \approx 3.6$ for MQ-137):

$$R_0 = \frac{R_{s,air}}{K_{air}} = \frac{R_L (V_c - V_{clean})}{K_{air} V_{clean}} \quad (5-7)$$

Concentration Extraction (PPM Calculation)

To calculate the ppm value, the sensor's response curve needs to be flattened out by mapping the resistance ratio R_s/R_0 to the gas concentration on a log-log scale. This allows the exact gas level to be calculated using a standard power formula:

$$C_{PPM} = a \left(\frac{R_s}{R_0} \right)^b \quad (5-8)$$

where:

- a is the scaling coefficient fitted to the logarithmic response curve.
- b is the exponent corresponding to the slope of the sensitivity curve on a log-log scale.



Figure 5-3. MQ-137 Ammonia Gas Sensor [28]

5.1.3. DHT22 Temperature and Humidity Sensor

The DHT22 is a digital temperature and humidity sensor. It combines temperature and relative humidity measurement in one device and provides a calibrated digital signal output. The sensor has a measurement range of -40°C to 80°C for temperature and 0–100% for relative humidity. It is itself calibrated by the manufacturer and has calibration coefficients stored internally as a program in the sensor's microcontroller. The device is designed for low power consumption, and the manufacturer has specified a signal transmission distance of more than 20 m under its stated conditions. The sensor uses a single-wire interface, making its connection to the microcontroller simple and convenient [29].

Working Mechanism

The DHT22 measures relative humidity using a capacitive polymer sensing element whose capacitance changes with the moisture content of the surrounding air. Temperature is measured using an NTC thermistor whose resistance varies with temperature. An

internal microcontroller processes these sensor signals, applies factory calibration, and converts them into digital values. The processed temperature and humidity readings are then transmitted to the ESP32 through a single-wire digital communication interface, minimizing signal noise and ensuring reliable environmental monitoring.



Figure 5-4. DHT22 Temperature and Humidity Sensor [30]

5.1.4. PTC Heating Element

The Positive Temperature Coefficient (PTC) heating element is a self-regulating ceramic heater integrated within the MQ-137 ammonia sensor. Unlike conventional resistive heaters, the resistance of a PTC material increases significantly as its temperature rises. This characteristic allows the heater to maintain a stable operating temperature without requiring external temperature control circuitry. The stable heating provided by the PTC element is essential for ensuring accurate and repeatable ammonia measurements.

Working Mechanism

When electrical power is applied, current flows through the PTC element, generating heat that raises the temperature of the SnO_2 sensing layer. As the temperature increases, the electrical resistance of the PTC material also increases, reducing the current and limiting further heating. This self-regulating behavior stabilizes the operating temperature and prevents overheating.

The relationship between resistance and temperature can be expressed as:

$$R = R_0[1 + \alpha(T - T_0)] \quad (5-9)$$

where R is the resistance at temperature T , R_0 is the reference resistance at temperature T_0 , and α is the temperature coefficient of the material. Although the actual resistance–temperature characteristic of a PTC heater is nonlinear, this equation illustrates the principle that resistance increases with temperature, enabling automatic temperature regulation.

5.1.5. LM2596 DC Step-Down Voltage Regulator

The LM2596 is a monolithic integrated circuit. It functions as a step-down voltage regulator and is capable of driving a 3 A load with good line and load regulation. The device is available in fixed output voltages of 3.3 V, 5 V, and 12 V, as well as an adjustable output version. It operates at a switching frequency of 150 kHz and requires a minimum number of external components, making it simple to use. For self-protection, it includes current limiting on the output switch and overtemperature protection [31].

Working Mechanism

The LM2596 operates by rapidly switching an internal transistor on and off, allowing energy to be stored in an inductor and released through a diode and output capacitor. A feedback circuit continuously monitors the output voltage and adjusts the PWM duty cycle to maintain a constant output despite changes in input voltage or load.

For an ideal buck converter, the output voltage is given by:

$$V_{out} = D \times V_{in} \quad (5-10)$$

where:

- V_{in} is the input voltage,
- V_{out} is the regulated output voltage,
- D is the PWM duty cycle (0–1).

By continuously adjusting the duty cycle, the LM2596 maintains a stable output voltage while achieving high conversion efficiency.

5.2. Software Implementation

The software implementation involves sensor data acquisition, environmental monitoring, TinyML model deployment, and ventilation control. The ESP32 DevKit V1 collects data from sensors, while the ESP32-S3 processes the data and executes the TinyML model. The model is trained offline using Python-based machine learning libraries and then converted into a lightweight format for deployment on the ESP32-S3.

5.2.1. Python

Python is a high-level programming language. It provides high-level data structures and supports object oriented programming. It focuses on simple syntax and readable code. It uses indentation to define blocks of code. Its programs are executed through the Python Interpreter. It supports dynamic typing so it is not required to declare the type of variable. It comes with a large collection of standard modules for different operations. Python can be extended with functions and modules implemented in other languages [32].

Role in the Developed System

Python was used to implement complete machine learning pipeline. Everything for the machine learning including data preprocessing, model training, performance evaluation and model export were done using Python. It is also used a primary language for developing the backend of the web-dashboard.

5.2.2. TensorFlow and Keras

TensorFlow is an end-to-end platform for machine learning and numerical calculations. It works with tensors and provides tools for building, training, evaluating and deploying machine learning models. It supports computational acceleration using GPUs and other hardware [33].

Keras is a high-level API used with TensorFlow to make building machine learning models faster and easier. It provides a simple way to create models and provides different functions to make the development process smooth [34]. TensorFlow provides underlying machine-learning framework while Keras provide a simple interface for developing ML models.

Role in the Developed System

TensorFlow and Keras were used to design, train, and evaluate the Multi-Layer Perceptron (MLP) model for predicting future temperature and ammonia concentration. After training, the model was converted into a lightweight format suitable for deployment on the ESP32-S3 microcontroller.

5.2.3. Pandas

Pandas is a Python library for data manipulation and analysis. The main data structures are DataFrame and Series. DataFrame allows data to be organized in rows and columns. It also provides data for cleaning, filtering, transforming and analyzing data. Pandas can read and write data in different formats such as CSV, Excel, JSON and others [35].

Role in the Developed System

For this project pandas was used to import dataset, perform data cleaning and generate time-series features. It was also used for organizing the processed data and prepare the data for training the machine learning model.

5.2.4. NumPy

NumPy is a Python library used for scientific and numerical computing. Its main feature is a multidimensional array which allows large amount of data to be processed efficiently. NumPy provides fast operations for arrays including mathematical operations, logical operations, sorting, shape manipulations etc. Many NumPy operations are performed using compiled code which makes them faster than performing equivalent operations using standard Python [36].

Role in the Developed System

NumPy was used to perform numerical calculations for data processing, feature engineering and normalization. It provided different mathematical tools and operations that were used throughout the machine learning pipeline.

5.2.5. Scikit-learn

Scikit-learn is a Python library that provides tools and functionalities for machine learning and data analysis. It supports supervised as well as unsupervised learning techniques. The library also provides preprocessing methods for preparing data, model-selection for comparisons and fine-tuning and evaluation metrics for assessing the model's performance. It is built on NumPy, SciPy and Matplotlib [37].

Role in the Developed System

Scikit-learn was used for feature normalization using StandardScaler. It also helped for dataset splitting and evaluation of the model using performance metrics like MAE, MSE, R^2 and other relevant metrics.

5.2.6. TensorFlow Lite for Microcontrollers

TensorFlow Lite for Microcontrollers is an inference framework designed to run ML models on microcontrollers. It accounts for constrained resources such as limited memory, processing power and energy availability for embedded devices. It allows trained models to operate directly on microcontrollers without relying on cloud for processing. Its design is suited to TinyML applications where small models are needed to operate on constrained hardware resources [38].

Role in the Developed System

TensorFlow Lite for Microcontrollers was used to deploy the trained MLP model on the ESP32-S3 allowing the model to inference locally from the data sent by the on ground sensors without cloud dependency.

5.2.7. PlatformIO

PlatformIO is a development platform for embedded systems and IoT applications. It provides a unified build environment and supports different microcontrollers and embedded frameworks. PlatformIO can manage tool chains, libraries and project dependencies while the platform.ini file is used to configure project settings. It also provides features like debugging and serial monitoring which is useful while developing and testing embedded systems [39].

Role in the Developed System

PlatformIO was used to develop the firmware for both the ESP32 DevKit V1 and ESP32-S3. All the libraries and dependencies were managed, and project settings were configured using this platform. It was also used for debugging and testing the developed system.

5.3. Dataset Analysis

5.3.1. Source and Purpose

The dataset used in this project was obtained from IEEE [20] and PLOS ONE [21]. The dataset consisted of environmental records from a poultry house during a commercial production cycle. It contains measurements of key parameters like temperature, relative humidity and ammonia concentration. The data was selected because it captured the real-world poultry house conditions rather than data collected under controlled laboratory settings. So the database captures the natural variations in temperature, humidity and ammonia concentration throughout the complete flock cycle. This makes the dataset suitable for developing machine learning models enabling the model to learn realistic environmental patterns and suitable for actual farm deployment.

5.3.2. Number of Samples

The dataset after cleaning and preprocessing contained 4,458 sensor observations covering data from approximately 34 days of farm conditions. After chronological splitting into training, validation, and test sets, the data looks like this:

Set	Rows	Purpose
Test	884	Final evaluation only, never touched during training
Validation	533	Threshold tuning, early stopping
Training (raw)	3,041	Model learns from this
Training (oversampled)	3,322	Fed to model.fit()

Table 5-1. Dataset split distribution

The data was divided into 80% for training and 20% for testing. The split was applied in chronological order to maintain the time sequence. Shuffling was avoided because it could allow the model to learn indirectly from future data points. The final 20% of the data was used as a test set to represent unseen conditions and were never used during training. The remaining data was then further divided for training and validation. Then the training data was oversampled to increase the proportion of rare events. Validation and Testing data were not oversampled at all and were left as it was.

5.3.3. Features and Target Variable

The dataset provides three primary sensor variables, from which the model's inputs and targets are derived.

Raw Sensor Variables

Variable	Unit	Sensor Source	Observed Range in Cleaned Data
temperature	°C	DHT22	10.2 to 33.8
humidity	% RH	DHT22	38.2 to 97.6
ammonia_ppm	ppm	MQ-137	0.0 to 31.7

Table 5-2. Raw sensor variables in the dataset

Model Input Features

The model used a total of 49 input features including the following feature types. These features allowed the model to capture the environmental trends for prediction.

Feature Type	Features
temperature	temperature_lag1, temperature_lag2, temperature_lag3, temperature_lag6, temperature_roll_mean3, temperature_roll_std3, temperature_delta1
humidity	humidity_lag1, humidity_lag2, humidity_lag3, humidity_lag6, humidity_roll_mean3, humidity_roll_std3, humidity_delta1
ammonia	ammonia_ppm_lag1, ammonia_ppm_lag2, ammonia_ppm_lag3, ammonia_ppm_lag6, ammonia_ppm_roll_mean3, ammonia_ppm_roll_std3, ammonia_ppm_delta1
Hour (sin)	hour_sin_lag1, hour_sin_lag2, hour_sin_lag3, hour_sin_lag6, hour_sin_roll_mean3, hour_sin_roll_std3, hour_sin_delta1
Hour (cos)	hour_cos_lag1, hour_cos_lag2, hour_cos_lag3, hour_cos_lag6, hour_cos_roll_mean3, hour_cos_roll_std3, hour_cos_delta1
Temp × Humidity Interaction	temp_hum_interaction_lag1, temp_hum_interaction_lag2, temp_hum_interaction_lag3, temp_hum_interaction_lag6, temp_hum_interaction_roll_mean3, temp_hum_interaction_roll_std3, temp_hum_interaction_delta1
Additional features	week: bird age, ammonia_accel: second-order ammonia acceleration, ammonia_ppm_lag9: 90-minute lag, ammonia_ppm_lag12: 120-minute lag, ammonia_ppm_roll_mean6: 6-step rolling mean (60 minutes), ammonia_ppm_roll_std6: 6-step rolling standard deviation (60 minutes), ammonia_ppm_trend_60min: 60-minute ammonia concentration trend

Table 5-3. Model input features

Target Variables

The model shares a common trunk which then bifurcates to different branches. Each branch is responsible for regression and classification outputs.

Head	Output	Type
t_reg (regression)	Predicted next temperature (°C)	Continuous
t_clf (classification)	Probability of temperature spike (<16 or ≥39°C)	0–1 probability
a_reg (regression)	Predicted next ammonia (ppm)	Continuous
a_clf (classification)	Probability of ammonia spike (≥15 ppm)	0–1 probability

Table 5-4. Model target variables

The regression outputs are the values which will be occurring 10 minutes later whereas the classification outputs are binary labels telling whether the next value exceeds the defined threshold.

5.3.4. Data Format

The original dataset was in a .xlsx file containing the environmental measurements. It was then converted into a CSV format for compatibility with Python-based ML models. The processed dataset contains the columns datetime, day_index, temperature, humidity, and ammonia_ppm which were used during model training.

5.3.5. Preprocessing

Before training the prediction model, the dataset underwent several preprocessing steps:

1. **Timestamp Cleaning and Resampling:** The raw sensor data contained duplicate timestamps and irregular time gaps. Some gaps lasted up to 22 hours. The duplicate readings were averaged, and the data was resampled to a uniform 10-minute interval. The small gaps were filled using interpolation resulting in 4,484 usable records for further processing.
2. **Time and Environmental Features:** Additional features were created from the cleaned data which included hour sine and cosine values to represent the daily time cycle. A temperature-humidity interaction feature, and week index were created to represent the approximate bird age. Finally, a 3-step rolling mean was also applied to temperature, humidity, and ammonia to reduce short-term sensor noise.
3. **Sliding-Window Feature Engineering:** Lag and rolling statistical features were generated using previous 10, 20, 30, and 60 minutes of data which included rolling mean, rolling standard deviation and changes between consecutive readings. Additional features were generated for ammonia due to its importance in detecting spikes. As a result, 49 input features were generated.
4. **Target Generation:** The next temperature and ammonia values were generated by shifting the smoothed value by one 10-minute step. The regression model was trained to predict the change in temperature and ammonia rather than predicting the absolute values. Similarly, classification labels were generated to indicate the potential environmental risks.
5. **Data Splitting and Normalization:** Data was divided into training 80% and the remaining 20% (884 records) into training sets chronologically to maintain the time sequence. The training pool was further divided into 3,041 training records and 533 validation records using a per-week time-based split. Input features and regression targets were normalized using the **StandardScaler** fitted only on the training data to prevent data leakage.
6. **Data Shuffling:** Only the training data was shuffled during model training. The validation and test data remained in chronological order to preserve the time-based nature of the dataset.
7. **Classification Threshold Calibration:** After training, the classification decision thresholds were adjusted using the validation data. A range of thresholds from 0.20 to 0.85 was tested. The threshold giving the best F1-score was selected separately for each classifier.

5.4. Model Architecture

The model developed and used for this project is a Bifurcated Multi-Task Sliding-Window Multi-Layer Perceptron (MLP). It is designed to predict temperature and ammonia values. Temperature changes are relatively smooth while ammonia changes can be sudden and could contain more sensor noise. Therefore, the network splits into two separate branches after some shared processing. One branch focuses on temperature while the other focuses on ammonia. Each branch performs both regression (predicting actual values in the future) and classification (detecting a hazardous condition).

The model uses a 49-feature input. Instead of just looking at the current sensor reading, it uses previous readings through a sliding window. It uses values from 10, 20, 30 and 60 minutes earlier, an additional 90 and 120 minutes specifically for ammonia to capture slower buildup trends and predict reliably. It also calculates rolling averages and standard deviations to show the trends and variability. Additional features include sine and cosine features for 24-hour daily cycle, temperature-humidity interaction, first-order changes ($\Delta 1$) to represent gas velocity, second-order changes ($\Delta 2$) to represent gas acceleration, and week index for the bird's age.

The standardized 49-feature input goes through a shared 64 neurons dense layer. ReLU activation is used for this. L2 Regularization and 20% dropout are used so that neurons are forced to learn the underlying patterns independently and also reduce overfitting. After this shared layer, the model splits into two branches. Temperature branch uses a 32 neurons ReLU layer, which predicts the next temperature value using linear activation and also predicts the possibility of any hazards using sigmoid activation. Ammonia branch uses a deeper network with 64 neurons and 20% dropout followed by 32 neurons, which directly produces the future ammonia concentration. An additional dedicated 16-neuron layer branches off this to produce the hazard classification output (whether ammonia exceeds the 15ppm threshold), giving the hazard classification decision its own extra capacity.

The model has both regression and classification outputs, so it needs to show the loss for both. Huber loss is used for regression outputs, whereas binary cross-entropy is used for classification outputs. Classification is given more weightage because detecting a dangerous condition is more important for safety. Both branches predict the change expected over the next 10 minutes, which is added to current reading to get the forecast, which performed better than predicting the absolute value directly. After training, the model is converted into TensorFlow Lite for deployment. It uses 8-bit Post-Training Quantization to make the model smaller. The final model is only 21.55 KB and is capable of low-latency inference on ESP32-S3, making it capable of making predictions locally.

5.5. Interfacing Technicalities and Protocols

The system uses different protocols and connections to function, and each connection has a specific purpose. Two microcontrollers and one server exchange data, and sensors send data to the sensor node microcontroller.

5.5.1. MQ-137 Communication Protocol and Data Transmission

The MQ-137 is an analog semiconductor ammonia sensor. It does not use digital communication protocols such as I²C, SPI, or UART. When the sensor is exposed to ammonia, the resistance of the sensing element changes. The sensor is connected to a voltage-divider circuit, which produces an analog voltage that varies with ammonia concentration. The analog voltage is read by the microcontroller through its Analog-to-Digital Converter (ADC). The ADC value is then processed to determine the sensor resistance and estimate the corresponding ammonia concentration in ppm using a calibration relationship.

5.5.2. DHT22 Communication Protocol and Data Transmission

The DHT22 uses digital communication for data transmission. It uses a single-bus, proprietary, timing-based protocol. The sensor typically has three wires: VCC, DATA, and GND. Data is transmitted from the DHT22 to the microcontroller as a 40-bit data frame. Of the 40 bits, 16 bits represent relative humidity, 16 bits represent temperature, and 8 bits form the checksum. The DHT22 communicates with the microcontroller using a single bidirectional data line on GPIO4, pulled up with a 10 k Ω resistor. The microcontroller reads the digital data through that pin, and the temperature (°C) and relative humidity (%RH) are then extracted from the received data.

5.5.3. RG1602A-IIC(P) LCD Communication and Data Transmission

The display is a 16×2 character LCD. It uses the I²C (Inter-Integrated Circuit) digital communication protocol for communication. The PCF8574T is the I²C interface expander for the display module. The module typically has four wires: VCC, GND, SDA, and SCL. SDA (Serial Data) and SCL (Serial Clock) are the communication lines. The microcontroller acts as the I²C master, while the PCF8574T acts as the I²C slave/interface expander. The PCF8574T converts the serial I²C data received from the microcontroller into parallel control/data signals for the LCD. The LCD itself typically uses a parallel interface, but the PCF8574T backpack allows it to be controlled through I²C using only two communication lines.

5.5.4. Fan Actuator

The fan uses two separate control signals. GPIO32 generates the PWM signal, which is responsible for controlling the fan speed. The PWM has 8-bit resolution, so the duty cycle can be represented using values from 0–255. Sometimes the fan may still rotate at 0% duty cycle because of leakage current. When GPIO27 is HIGH, the fan can operate, and when it is LOW, the fan is off. Therefore, the PWM controls the speed of the fan.

5.5.5. Heater Actuator

The PTC heater only needs to be switched ON or OFF. GPIO25 is connected to the IN pin of the relay module, and the relay acts as a digital switch for the heater. The relay module is used in active-LOW mode, so the heater is turned ON when GPIO25 is LOW and turned OFF when GPIO25 is HIGH. The heater is controlled by the heater control value sent from the master node.

5.5.6. Sensor Node to Master Node

The sensor node uses an ESP32 DevKit V1. It reads the DHT22 sensor on GPIO4 and reads analog values from the MQ-137 sensor on GPIO34. These readings do not go straight to the server. The node packs this data into a C struct called SensorPacket, which holds the sequence number and sensor values. The node sends this struct via ESP-NOW to the master's MAC address every five seconds. ESP-NOW uses the standard 2.4 GHz radio and skips Wi-Fi association and IP addressing, keeping things simple for two boards sitting close together. The system copies the struct directly into the payload and checks the delivery status at the radio layer.

5.5.7. Master-to-Backend Link

The master node (ESP32-S3) is responsible for communicating with the Django backend. It uses Wi-Fi for communication with the server. The master needs to find the Django server's IP address, which it does using UDP discovery. The master sends a small UDP broadcast message containing an identification string to find the server. The Django server listens for this message and sends a response back. The master uses the source IP address of the response to identify the server. It attempts to discover the server up to 8 times, every 1.5 seconds. After finding the server, the master communicates with Django using HTTP/1.1. Sensor and prediction data are combined and converted into JSON format, and this JSON data is sent to Django using an HTTP POST request on port 8000. Django processes the data and sends the result back in JSON format, and the master then sends this result to the sensor node using ESP-NOW. There is also a separate feature that regularly checks Django for manual controls or dashboard overrides; sensor communication and manual dashboard commands can work independently.

6. RESULT AND ANALYSIS

The system manages to read the raw sensor data from MQ-137 and DHT22 and as the data is recorded, our machine learning model will calculate the predicted values of ammonia and temperature. The system produces output of predicted values of ammonia and temperature and decides which actuator to run based on the predicted values. The system also has a protective layer where, in case ammonia spikes rapidly and our model does not detect it, the actuator runs automatically.

6.0.1. Distribution of Input Variables

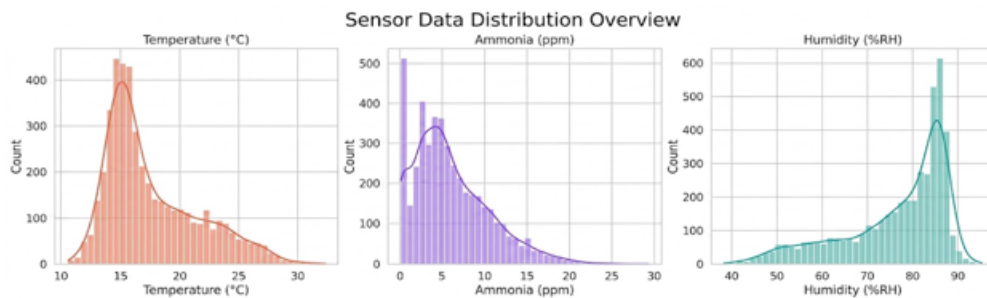


Figure 6-1. Distribution of temperature, humidity and ammonia concentration

The figure presents the distribution of three profiles. The collected temperature data ranges from 11°C to 32°C. Most of the temperature readings are concentrated around 14°C–16°C. The temperature changes are described to be smooth and continuous.

The humidity data is left-skewed. A large portion of the data falls between 80% and 88% relative humidity, indicating a consistently damp environment.

The ammonia data is heavily right-skewed. Most of the ammonia readings are below 6 ppm. Hazardous readings above 15 ppm occur but are less frequent. So the high ammonia events are rare events compared with normal readings. This creates an imbalance affecting the ammonia hazard classification.

6.0.2. Correlation Analysis

The correlation matrix presented in Figure 6-2 illustrates the relationship among the measured environmental variables.

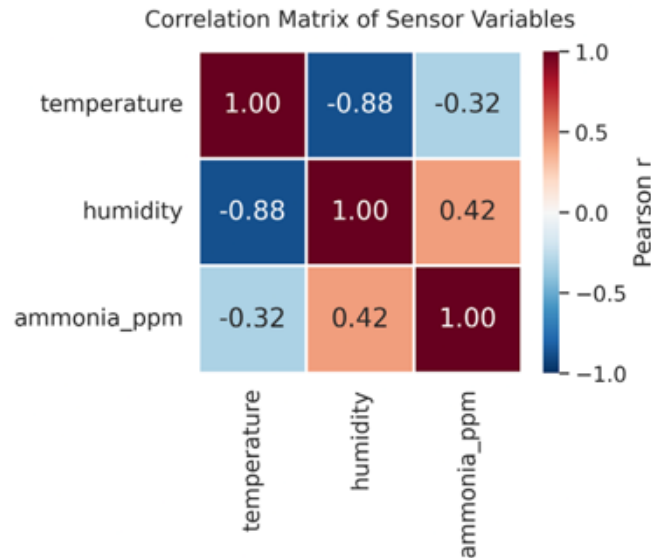


Figure 6-2. Correlation matrix of environmental variables

From the correlation matrix, temperature and humidity exhibited a strong negative correlation of -0.88. It indicates that an increase in temperature was generally associated with a decrease in relative humidity. Humidity and ammonia concentration show a moderate correlation of 0.42, suggesting that humidity may influence ammonia concentration. However, additional environmental variables would be required to establish a direct causal relationship.

Temperature and ammonia concentration exhibit a weak negative correlation ($r = -0.32$), indicating that temperature alone provides limited predictive information for ammonia concentration. Overall, the correlation analysis demonstrates that temperature and humidity are strongly related while ammonia concentration appears to be influenced by additional factors.

6.1. Regression Model Performance

Regression models were developed to predict the subsequent temperature and ammonia concentration using historical sensor observations.

6.1.1. Actual vs Predicted Values

The comparison between actual and predicted values is presented in Figures 6-3 and 6-4.

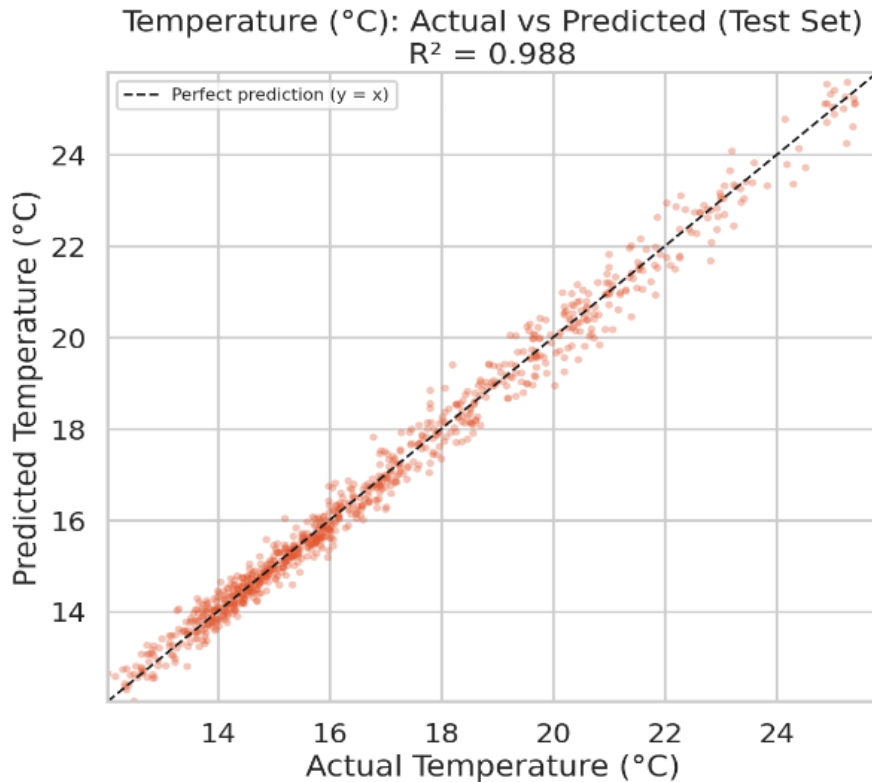


Figure 6-3. Actual vs predicted temperature on test set

From the above scatter plot showing actual vs predicted temperature on test set, the points are clustered very tightly around the diagonal line across the entire range. This indicated the strong agreement between the predicted and actual temperature values. The regression model achieved an R^2 of 0.988, meaning it explains 98.8% of the variance in the observed temperature values. A few points farther from the line indicated occasional prediction errors, but relatively limited. This indicated that the model is capable of accurately learning the environmental temperature variations.

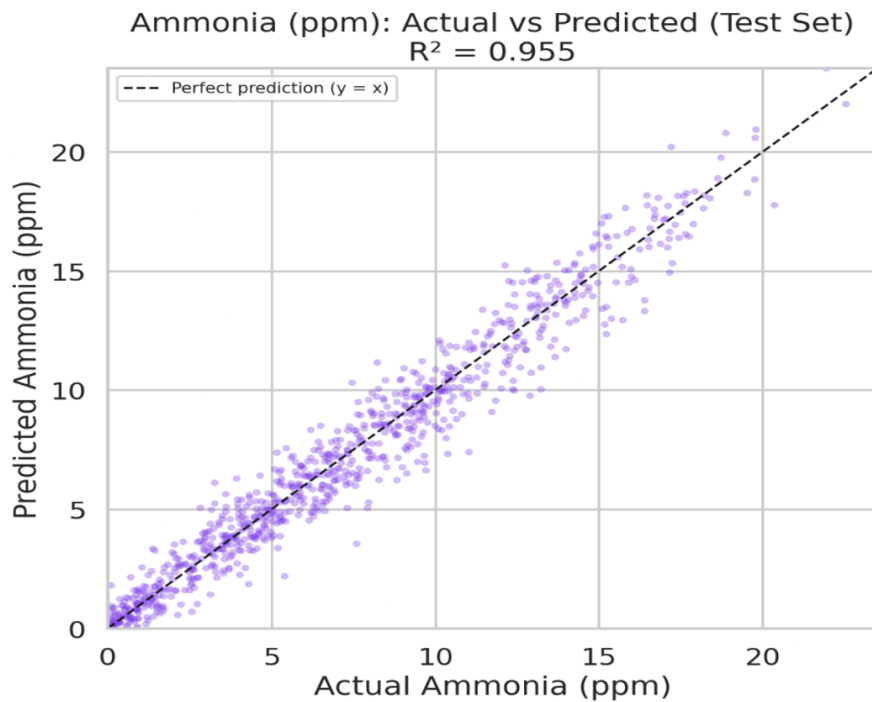


Figure 6-4. Actual vs predicted ammonia on test set

Figure 6-4 shows the actual and predicted ammonia concentration. The model achieved an R^2 value of 0.955, demonstrating a strong relationship between predicted and actual ammonia values. On observing the plot, the data points are generally distributed around the perfect-prediction line. A greater degree of scattering can be observed compared with the temperature prediction. This indicates that ammonia concentration is comparatively difficult to predict accurately.

6.1.2. Residual Analysis

Residual plots for both regression models are presented in Figures 6-5 and 6-6.

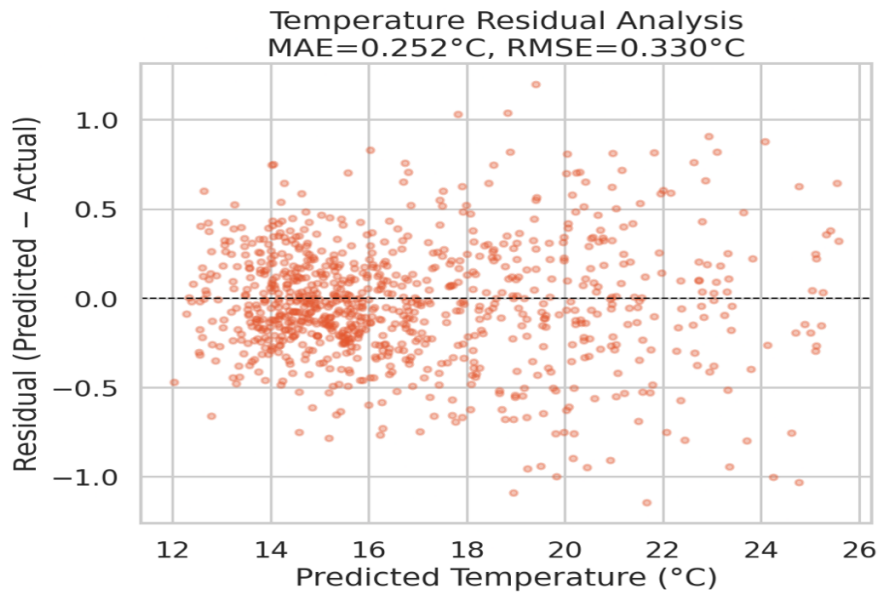


Figure 6-5. Residual plot for temperature prediction

The temperature residual analysis further supports the performance of the model. The obtained MAE of 0.252°C indicates the average absolute difference between predicted and actual temperature values was approximately 0.252°C. Similarly, the RMSE of 0.330°C indicates the overall prediction error remains relatively low. Most residuals were distributed around zero, indicating that the model does not exhibit a strong tendency to overpredict or underpredict temperature.

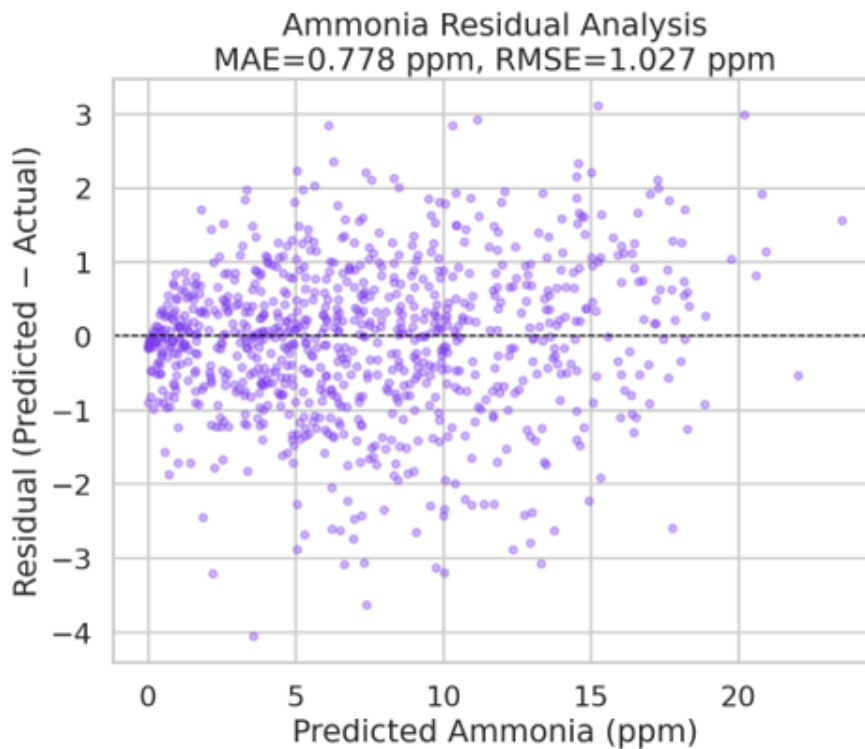


Figure 6-6. Residual plot for ammonia prediction

The ammonia residual gave an MAE of 0.778 ppm and an RMSE of 1.027 ppm. Most residuals remained centered around zero, showing that the predictions were centered around actual values. However, larger positive and negative residuals were present, particularly at higher ammonia concentration. This greater variation may be associated with the more dynamic nature of ammonia concentration. This is mainly affected by the ventilation conditions, poultry activity, and waste accumulation. The scattering observed may be partly associated with the distribution of the dataset. The dataset contained a larger number of readings at lower concentrations, with fewer readings at higher concentrations.

6.1.3. Error Distribution

The prediction error distributions are illustrated in Figure 6-7.

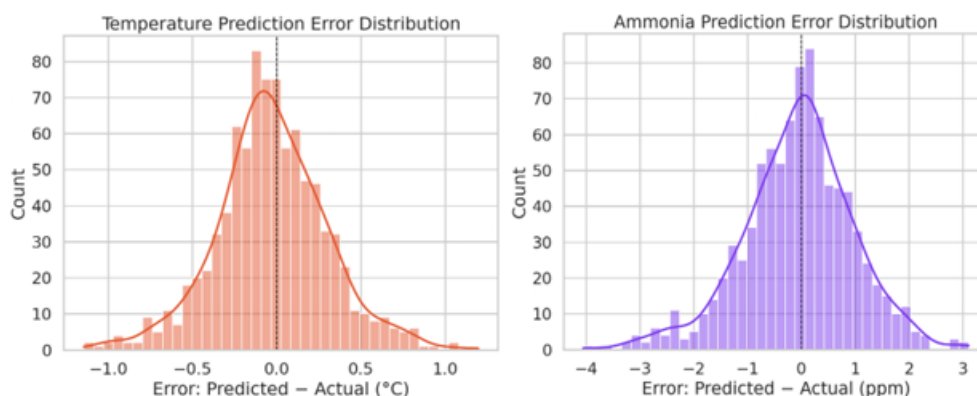


Figure 6-7. Prediction error distribution for temperature and ammonia

The prediction error distributions for temperature and ammonia are shown in corresponding histograms. The error is calculated as the difference between the predicted and actual values. The positive errors indicate overprediction and negative errors indicate underprediction.

For temperature, the error distribution is highly concentrated around zero, with most prediction errors approximately $\pm 0.5^{\circ}\text{C}$. The distribution is slightly shifted toward the negative side, suggesting a small tendency toward underprediction. For ammonia, the error distribution is wider, with error extending approximately from -4 to 3 ppm at lower concentration. This shows greater variability and a longer negative tail compared with temperature error distribution.

This observation is consistent with a low MAE of 0.252°C and RMSE of 0.330°C for temperature prediction. The higher error variability is consistent with the MAE of 0.778 ppm and RMSE of 1.027 ppm for ammonia.

6.1.4. Performance Metrics

The overall regression performance is summarized in Table 6-1.

Metric	Temperature ($^{\circ}\text{C}$)	Ammonia (ppm)
MAE	0.252	0.778
MSE	0.109	1.054
RMSE	0.330	1.027
R^2	0.988	0.955

Table 6-1. Overall regression performance on the test set

The results demonstrated the performance of the MLP regression model on the test set for temperature and ammonia prediction. Both models demonstrated strong predictive performance, achieving an MAE of 0.252°C, an MSE of 0.109, and an RMSE of 0.330°C, with an R^2 of 0.988 for temperature. For ammonia prediction, the model obtained an MAE of 0.778 ppm, an MSE of 1.054, and an RMSE of 1.027 ppm, with an R^2 of 0.955. This showed that the temperature prediction has higher predictive accuracy than ammonia, as indicated by their high R^2 values. Overall, the results confirmed that the MLP effectively captured the pattern required for predicting both temperature and ammonia.

6.1.5. Feature Importance Analysis

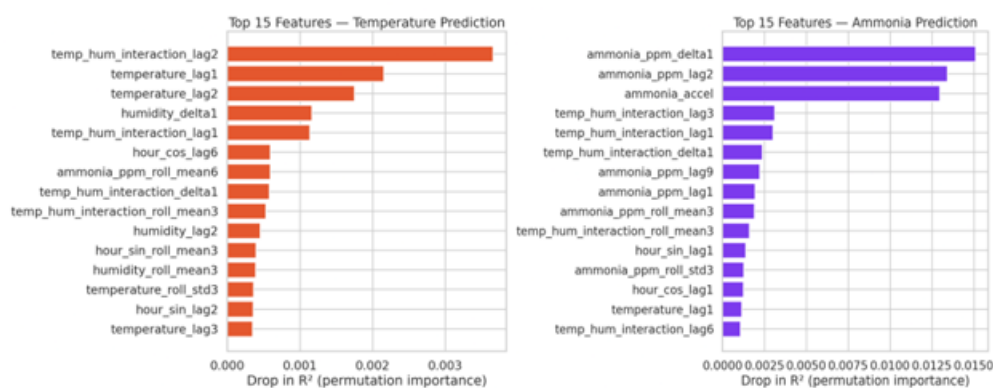


Figure 6-8. Permutation importance for temperature and ammonia concentration

The temperature prediction model shows that the most influential feature is `temp_hum_interaction_lag2`, which means the interaction of temperature and relative humidity 20 minutes ago. The importance of `temperature_lag1` and `temperature_lag2` demonstrates a strong temporal dependency in temperature. In other words, recent temperature observations are highly useful for predicting temperature values. The ammonia prediction model exhibits a noticeably different feature-importance pattern. The three dominant features, `ammonia_ppm_delta1`, `ammonia_ppm_lag2`, and `ammonia_accel`, have larger permutation importance. The high importance of `ammonia_ppm_delta1` indicates that a recent change in ammonia concentration is the strongest predictor for the target ammonia level.

Additional ammonia and temperature related features such as `ppm_lag`, `roll_mean`, and `roll_std` provide further predictive information.

6.2. Classification Performance

6.2.1. ROC Curve

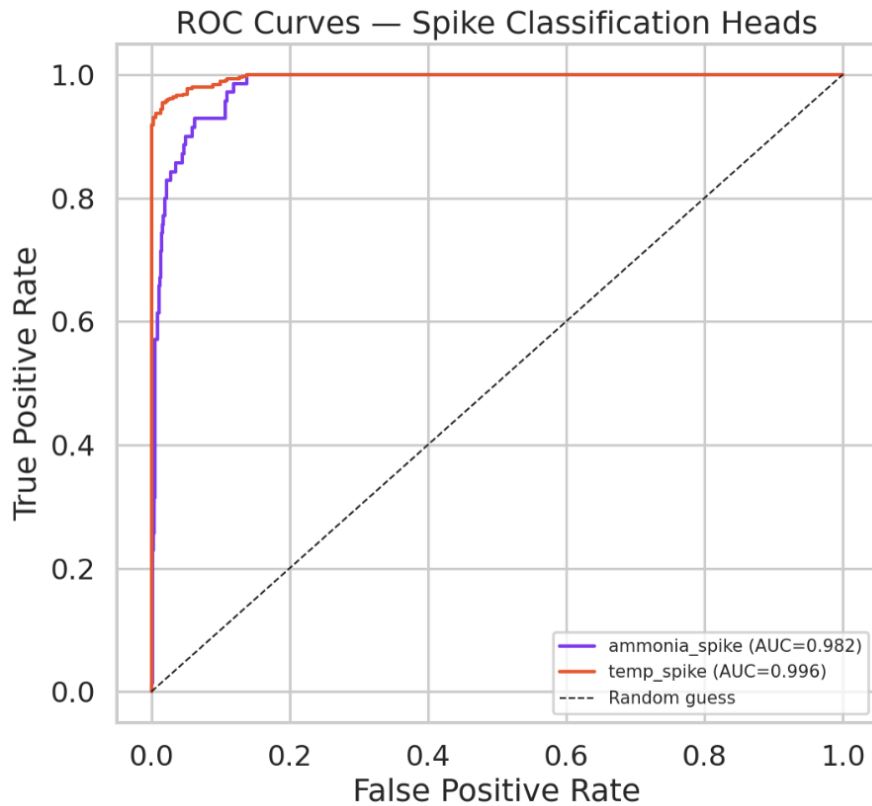


Figure 6-9. ROC curve

ROC analysis was performed on the model using the test dataset. It checks how classification performance changes when the threshold is changed. AUC (Area Under the Curve) is used to measure how well the model separates the two classes across different thresholds. The temperature head has an AUC of 0.996 and the ammonia head has an AUC of 0.982. Both values, being very close to 1.0, indicate strong discrimination between normal and hazardous conditions. The ROC curves move steeply toward the upper-left region, indicating high true-positive rates while keeping the false-positive rates relatively low. This confirms that the model can be reliably used as an automatic safety trigger in the control system.

6.2.2. Confusion Matrix

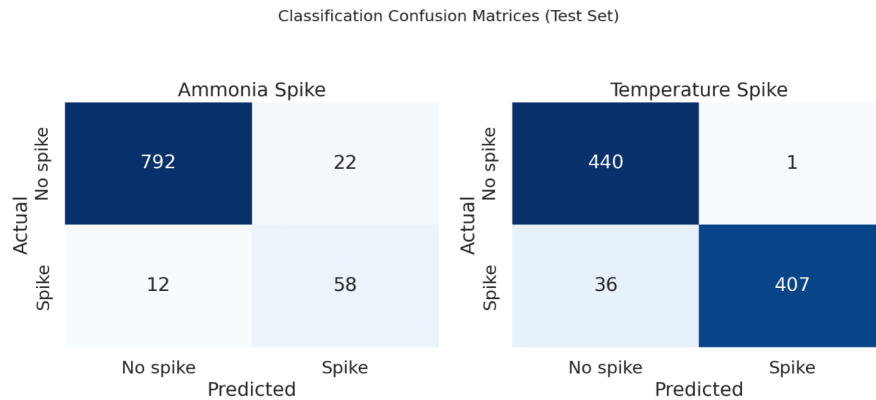


Figure 6-10. Confusion matrix

Ammonia Classification Head:

The ammonia classification head achieved an overall accuracy of 96.2%, a precision of 72.5%. Similarly, a recall of 82.9% was achieved with F1-score = 0.773. The model correctly detected 58 out of 70 actual hazardous ammonia spikes (TP), with only 12 missed hazardous events (FN). There were 22 false positives (FP), which means the model generated false alarms 22 times across 814 normal baseline periods. Specificity was 97.30%, which means the model correctly recognized most normal conditions as non-hazardous.

Temperature Classification Head:

The temperature classification head achieved an overall accuracy of 95.8%, a precision of 99.8%. Similarly, a recall of 91.9% was achieved with F1-score = 0.957. There was only 1 false positive (FP) among 441 normal operating states, which means the model rarely generates a temperature hazard alarm during the normal conditions without inducing unnecessary actuator switching.

Metric	Ammonia spike (≥ 15 ppm)	Temperature spike (outside 16–39°C)
Accuracy	0.962	0.958
Precision	0.725	0.998
Recall	0.829	0.919
F1	0.773	0.957

Table 6-2. Spike classification performance metrics (test set)

6.3. Discussion

The evaluation results demonstrate that the Edge-AI based Predictive Closed Control System for Smart Poultry Farming effectively predicts the future hazards while continuously monitoring important environmental parameters such as temperature, humidity and ammonia concentration using sensors such as MQ-137 and DHT22, maintaining suitable environmental conditions for broilers. The model shows consistent improvements in key metrics, confirming its effectiveness in real-world scenarios.

The model achieves an R^2 value of 0.988 for temperature, indicating that the model is able to capture approximately 98.8% of the variation observed in the temperature data, demonstrating a strong relationship between the predicted and actual temperature values. Similarly, the model achieves an R^2 value of 0.955 for ammonia prediction, indicating that the model explains approximately 95.5% of the variation observed in the ammonia concentration data. These results indicate that the system can effectively capture changes in both ammonia and temperature values in the monitored environment. The R^2 values suggest that the model's predictions closely follow the patterns observed in the actual sensor measurements.

MAE and MSE show a considerable improvement by decreasing up to 0.252 and 0.109, respectively, for temperature. Similarly, for ammonia the value is around 0.778 and 1.054.

This confirms that the developed system still requires further improvement for the ammonia prediction. Our work reflects that the fault is not with the model and the hardware; the problem or limitation faced highly due to inconsistency in the value of ammonia in the dataset, which was accessed from IEEE DataPort [20] and PLOS ONE [21].

The integration of machine learning further improves the developed system by providing the possibility of predicting unfavorable environmental conditions before it becomes critical. The result from the model suggests that ML can be useful for identifying patterns in environmental data and supporting early decision-making. However, the prediction of the model strongly depends on the quality, quantity and diversity of the trained data. Environmental parameters have a direct relation with poultry health and productivity. Excessive ammonia, inappropriate temperature and humidity increase the risk of disease and mortality. Therefore, continuous monitoring and early detection of those abnormal conditions can contribute to better poultry-house management.

Compared with conventional manual monitoring, the developed system provides continuous and automated observation of the poultry environment through a real-time

dashboard. This can reduce the need for frequent manual measurement and allows the farmer to react more quickly to unfavorable conditions. The combination of real-time sensing, automated control and machine learning prediction makes the system more suitable for proactive poultry farm management.

Despite these advantages, the developed system has some limitations. Sensor readings may be affected by placement, environmental interference and hardware limitations. Similarly, the machine learning model requires a sufficiently large dataset to achieve reliable predictions. Network connectivity and power availability may also affect the continuous operation of the IoT system.

Overall, the findings demonstrate that the developed IoT and machine-learning-based approach can provide an effective foundation. With further testing using a larger real-world environment and dataset, improved sensor calibration, additional environmental and health parameters with longer field experiments, the system could further be improved to provide more accurate control and prediction.

6.4. Result of Overall System

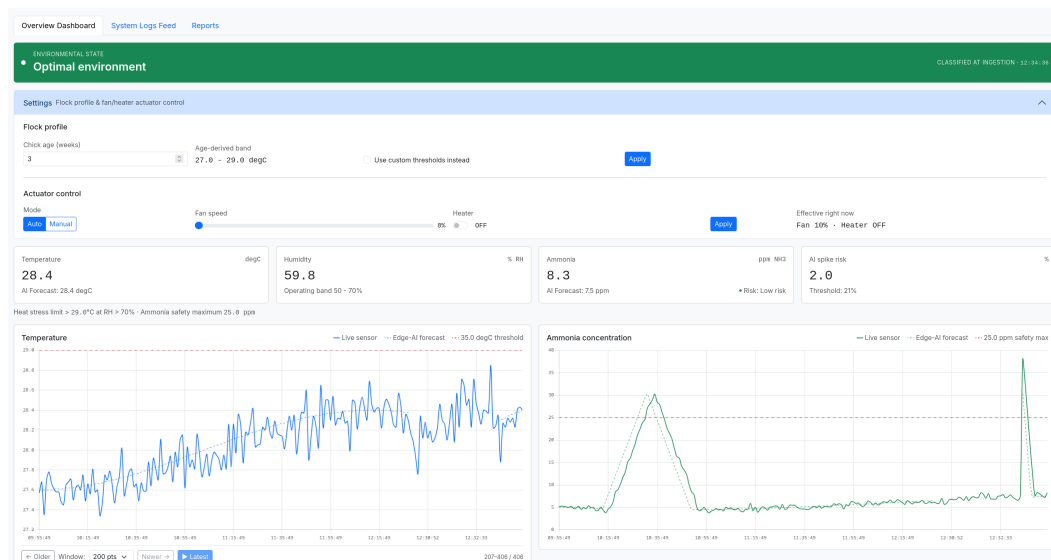


Figure 6-11. Web dashboard showing live sensor readings, Edge-AI forecasts, and the console feed

The overall system demonstrated the successful integration of environmental sensing, Edge-AI prediction, processing and actuator control for Smart Poultry Farming. The system continuously monitored the important parameters: temperature, humidity and ammonia concentration. The dashboard successfully displayed real-time environmental parameters from the sensing system. The dashboard presents AI forecast values along with live readings. During the observed operation, the system recorded a temperature of 28.4 °C, humidity of 59.8% RH, and ammonia concentration of 8.3ppm. Along with the current sensor readings, the dashboard displayed the AI forecast representing the predicted upcoming values. The dashboard also classified the current environmental condition as Optimal Environment and indicated an AI spike risk of 2% which is below the threshold. The graphs provided a comparison between the actual sensor measurement and predicted values. This allowed the prediction behavior to be visually observed over time. In addition, the actuator-control section shows the system operating in automatic mode with the fan running at 10% and heater turned OFF. Overall the results demonstrate that the developed system successfully combined real-time monitoring, next value prediction using Edge-AI, risk assessment and automated actuator control.

7. FUTURE ENHANCEMENTS

7.1. Additional Environmental Parameters

The current model uses only temperature, humidity, and ammonia. Additional parameters such as CO₂ concentration, fan speed, airflow, and feed/water consumption would make the prediction model more comprehensive. CO₂ concentration indicates the effectiveness of ventilation and the respiratory activity of the birds, and its change is associated with changes in air quality. Adding CO₂, fan speed, and airflow would help determine the ventilation condition, while feed and water consumption would provide information about flock health and activity.

7.2. Diverse and Larger Dataset

The developed model can be improved by collecting data from more poultry houses, across different seasons, flock sizes, and ventilation conditions. A more diverse dataset would allow the model to learn a wider range of farming practices and environmental conditions, helping it generalize better across different poultry farms.

7.3. Disease and Behavior Monitoring

A future version integrated with microphones and cameras could detect abnormal poultry activity, feeding behavior, sound, and other indicators of disease and stress. Audio data could be analyzed to identify abnormal vocalizations, while camera-based monitoring could be used to detect changes in movement and feeding behavior.

7.4. Enhanced Sensor Technology

The current system uses relatively simple sensors, such as the MQ-137 and DHT22, which have limitations in terms of accuracy, power consumption, and sensing range. In the future, more accurate and energy-efficient sensors could be used to obtain better-quality environmental data, improving the performance and prediction accuracy of the system.

8. CONCLUSION

The project accomplished its primary objective: to design and develop an IoT-based Edge-AI system for real-time monitoring, prediction and autonomous control of poultry house environmental conditions. The system integrates temperature, humidity and ammonia sensing with ESP32-based processing, wireless communication through ESP-NOW, a TinyML-based prediction model and actuators for environmental control. In addition, a web-based dashboard was developed to provide real-time visualization of sensor measurements, predicted values, system status and alerts, allowing farm environments to be monitored remotely.

The MLP model utilizing TinyML enables the prediction of environmental parameters directly at the edge. The system uses the regression model to learn the relationship between historical environmental conditions and predict future temperature and ammonia values. In addition, the predicted values can be used to identify potential changes or spikes in environmental conditions and support appropriate control decisions. The model performance in the present system indicates a moderate classification performance and provides scope for future improvement through a diversified, larger dataset and better sensor calibration.

The integration of Edge-based prediction with automated actuators allows the system to operate as a predictive closed-control system rather than a monitoring system. Overall, the developed system demonstrates the feasibility of combining IoT, Edge-AI, TinyML, automated control and web-based monitoring for smart poultry farming. The developed system provides a foundation for intelligent livestock environmental management. Future improvements can focus on increasing the diversity and size of the dataset, using enhanced sensor technology, and integrating systems for disease and behavior monitoring.

A. APPENDICES

A.1. Project Timeline

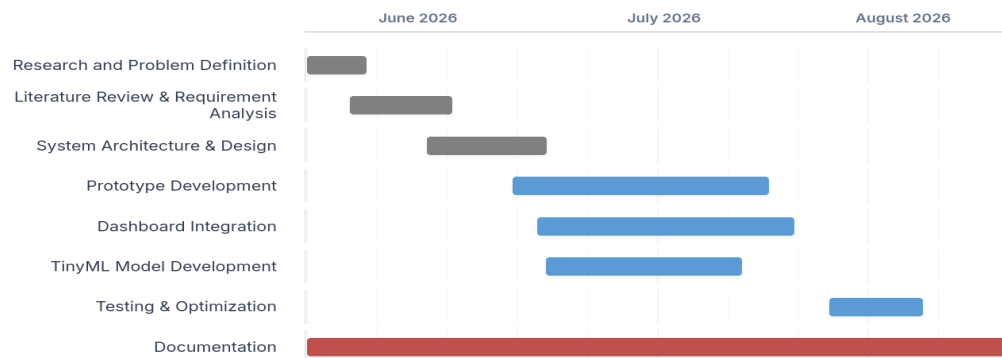


Figure A-1. Project Timeline Gantt Chart

A.2. Bill of Materials (BOM)

SN	Components	Qty	Unit Cost (NPR)	Total (NPR)
1	ESP32-S3 Microcontroller (Master Node)	1	1500	1500
2	ESP32 DevKit V1 (Slave Node)	1	750	750
3	DHT22 Temp & Humidity Sensor	1	350	350
4	MQ-137 Ammonia (NH ₃) Gas Sensor	1	2000	2000
5	Relay Module & Passive Components	1	150	150
6	DC Cooling Fan (12V, 4-Pin)	1	250	250
7	PTC Heating Element (12V)	1	350	350
8	12V DC Power Supply Adapter	1	500	500
9	LM2596 DC-DC Buck Converter Module	1	150	150
10	Puff Board for Prototyping	2	250	500
11	Connectors, Headers & Wiring Harness	1	200	200
12	Protective Hardware Enclosure Box	1	1000	1000
			Total	7700 NPR

Table A-1. Project Budget

A.3. Circuit Diagram

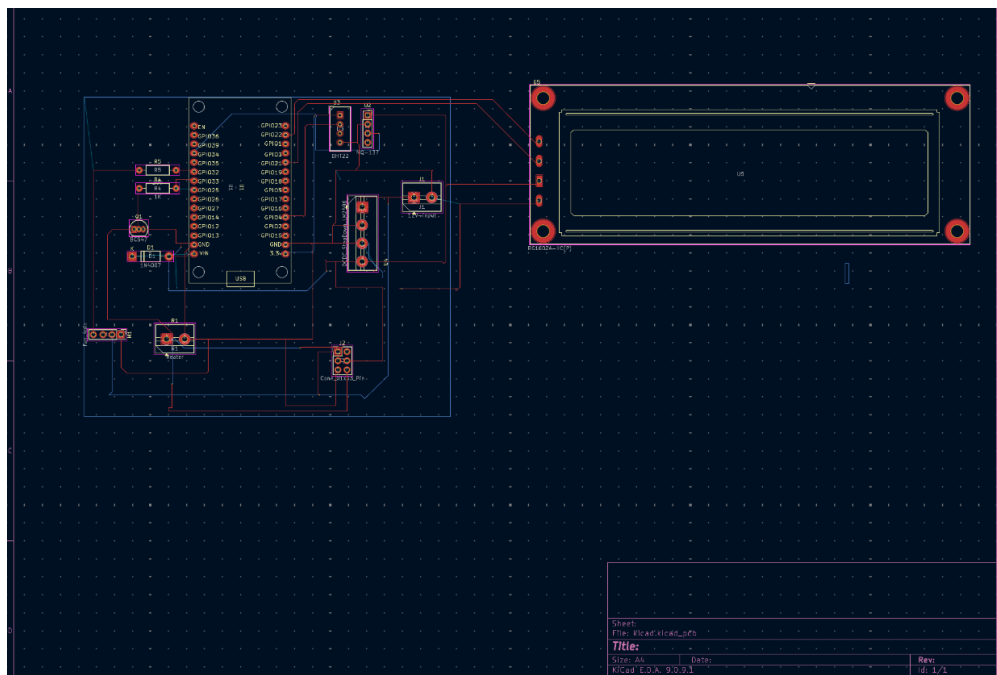


Figure A-2. Complete system schematic showing all subsystem interconnections

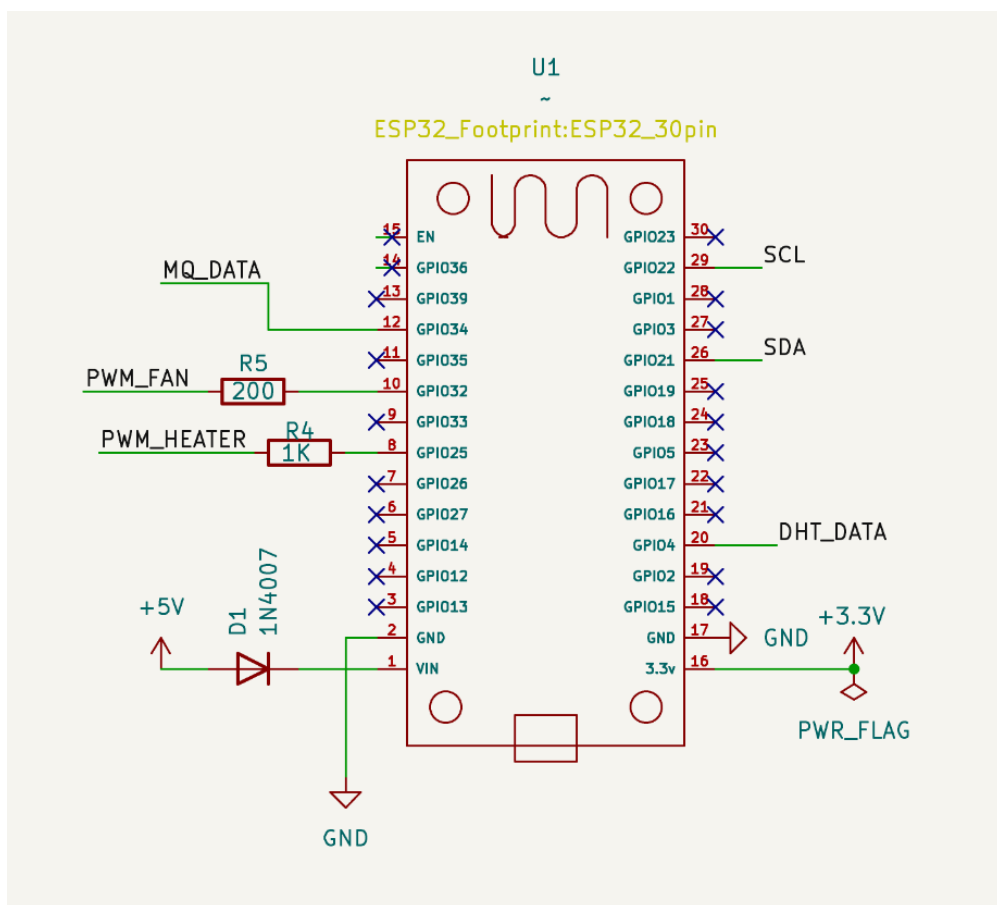


Figure A-3. ESP32 configuration circuit used for sensor interfacing and data acquisition

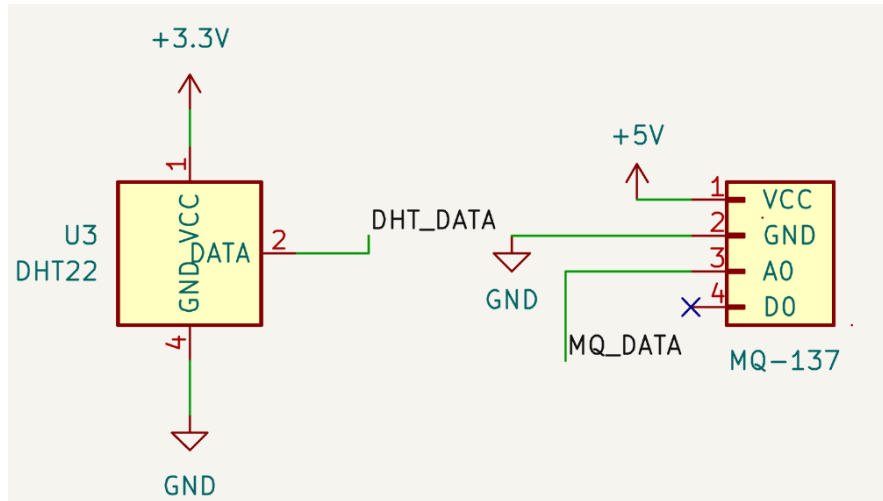


Figure A-4. Sensor interfacing circuit showing DHT22 and MQ-137 connections

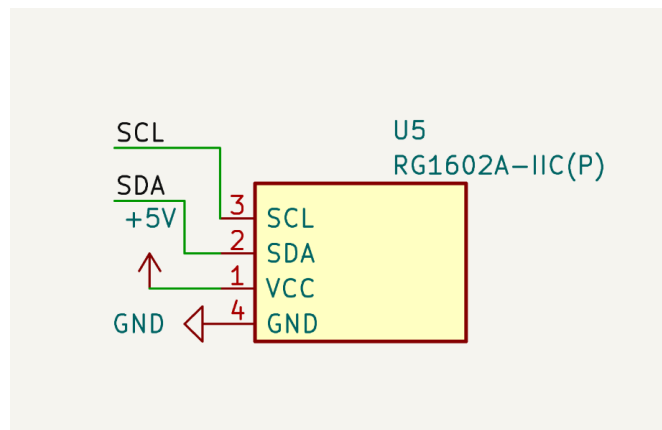


Figure A-5. RG1602A I2C LCD display interfacing circuit

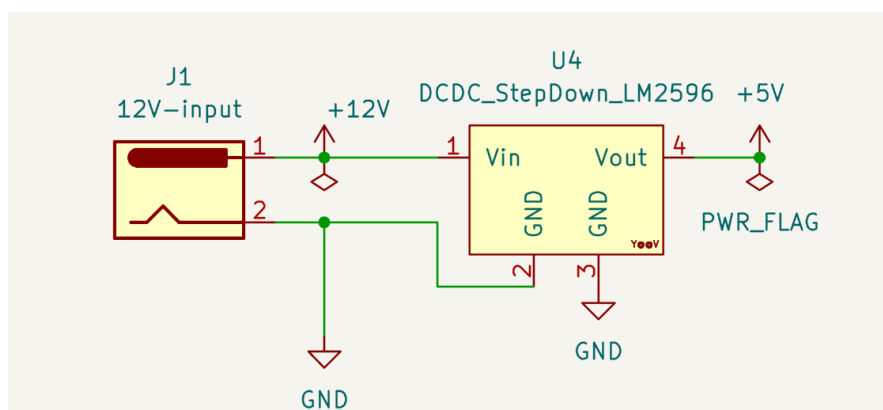


Figure A-6. Power supply circuit for the portable hardware system

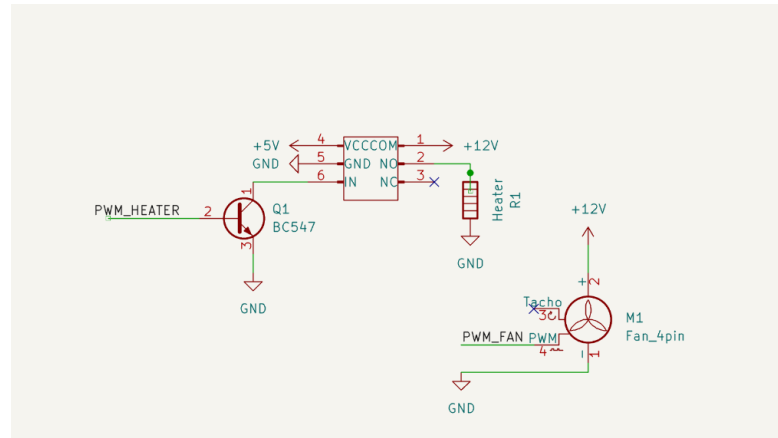


Figure A-7. Actuator control circuit showing fan and heater connections

A.4. PCB Diagram

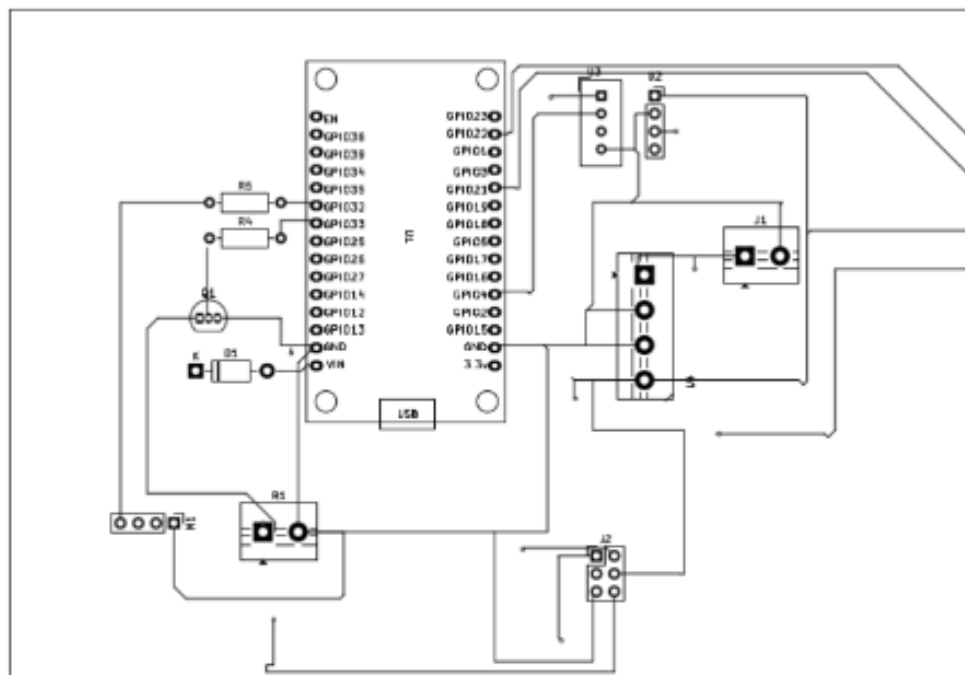


Figure A-8. PCB front layout design including relay driver and LCD display placement

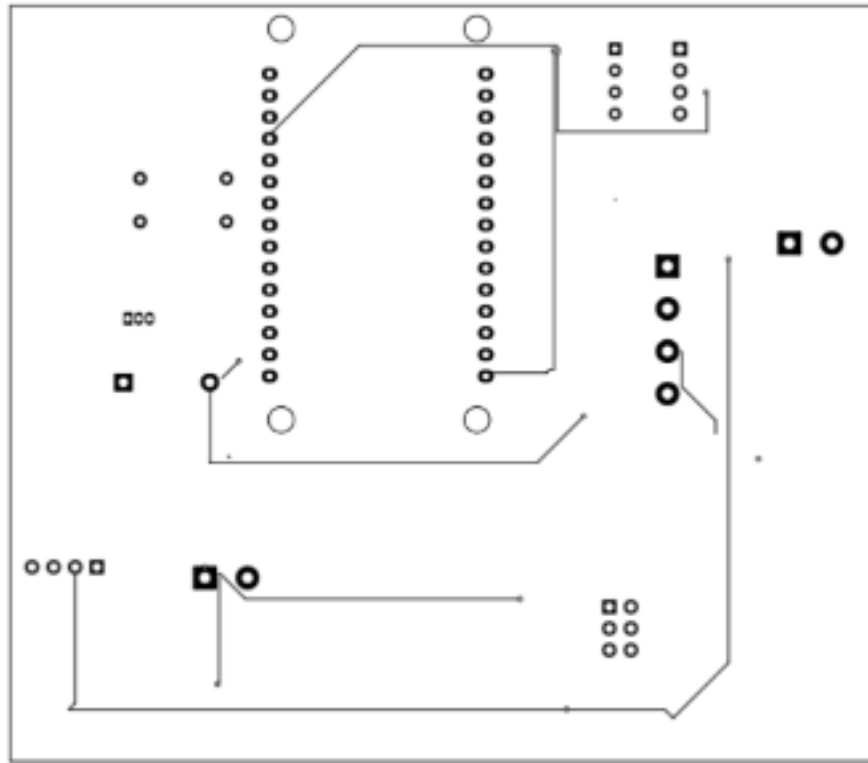


Figure A-9. PCB back layout design for the smart poultry monitoring system

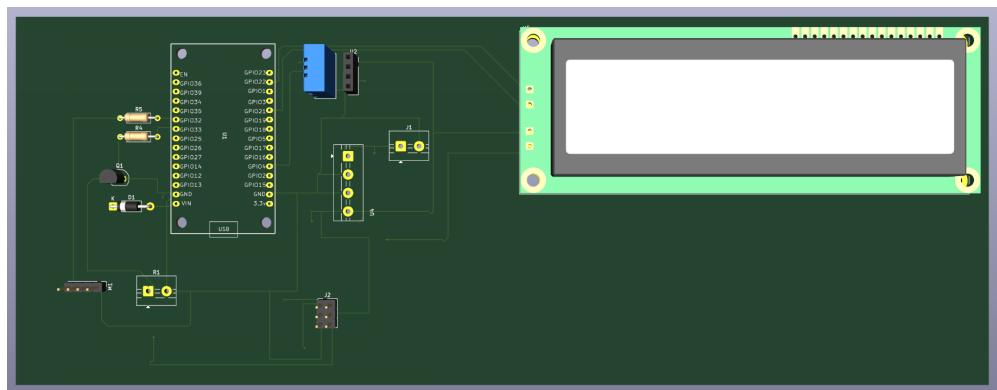


Figure A-10. 3D rendered view of the final assembled PCB with LCD display

A.5. Enclosure Design

The hardware case was develop for the PCB, sensors and cable as well to create ventilation for cooling process. The case contains a pannel enable air circuiation inside the case.

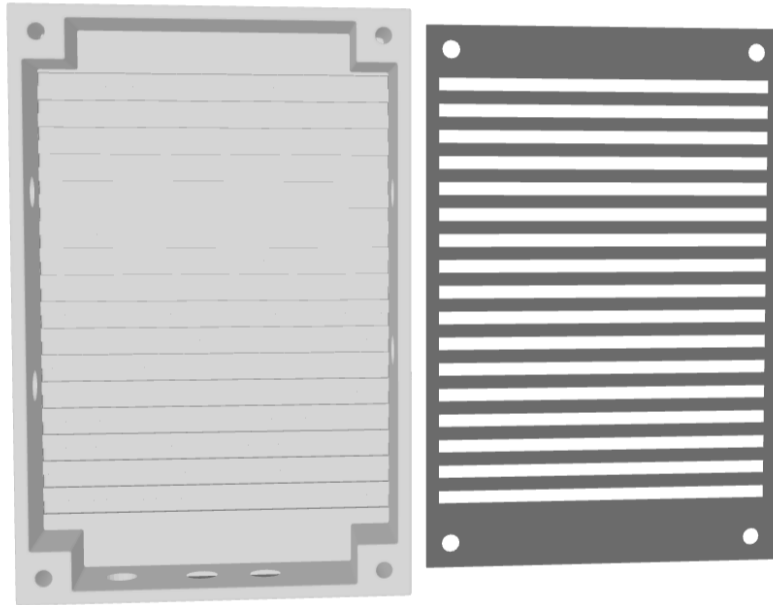


Figure A-11. 3D rendered design of the protective hardware enclosure box showing the base panel and ventilated front grille

A.6. Hardware Interconnection Table

To ensure strict microsecond-level timing and accurate data acquisition, the physical wiring between the sensors, actuators, display, and the microcontroller is mapped as follows:

Source Component	Pin Name	Target Component	ESP32 GPIO	Logic / Signal Type
DHT22	DATA	ESP32 DevKit V1	GPIO4	Single-wire Data
DHT22	VCC	ESP32 DevKit V1	3.3V	Power Supply
DHT22	GND	ESP32 DevKit V1	GND	Ground
MQ-137	A0	ESP32 DevKit V1	GPIO34	Analog Input (ADC)
MQ-137	VCC	ESP32 DevKit V1	5V	Power Supply
MQ-137	GND	ESP32 DevKit V1	GND	Ground
MQ-137	DO	ESP32 DevKit V1	-	Digital Output (Not Used)
LCD Display (RG1602A-IIC)	SCL	ESP32 DevKit V1	GPIO22	I2C Clock
LCD Display (RG1602A-IIC)	SDA	ESP32 DevKit V1	GPIO21	I2C Data
LCD Display (RG1602A-IIC)	VCC	Power Supply	5V	Power Supply
LCD Display (RG1602A-IIC)	GND	ESP32 DevKit V1	GND	Ground
Fan (4-Pin)	PWM	ESP32 DevKit V1	GPIO32	PWM Output
Fan (4-Pin)	Tacho	ESP32 DevKit V1	-	Tachometer Signal (Not Used)
Fan (4-Pin)	VCC	Power Supply	12V	Power Supply
Fan (4-Pin)	GND	Power Supply	GND	Ground
Relay Module	Base (Q1 BC547)	ESP32 DevKit V1	GPIO25	Switching Control (via R4 1k Ω)
Relay Module	IN	Q1 BC547 (Collector)	-	Relay Trigger Signal
Relay Module	VCC	Power Supply	5V	Power Supply
Relay Module	GND	Power Supply	GND	Ground
Relay Module	COM	Power Supply	12V	Switched Power Input
Relay Module	NO	Heater Element	12V	Switched Power Output
Heater Element	GND	Power Supply	GND	Ground

Table A-2. System Hardware Interconnection Map

A.7. Dataset

A.7.1. Poultry Farm Environmental Sensor Dataset (IEEE Dataport)

[20]

Primary Reference: Ajay Barsagade, Poultry farm Environmental sensor Dataset, IEEE Dataport, 2023.

Data Format: Raw IoT and microclimate time-series telemetry provided in tabular (.csv / .xlsx) format. The records encompass multi-parameter indoor environmental readings including ambient temperature, relative humidity, ammonia (NH₃) gas concentration, carbon dioxide (CO₂), and light intensity logged across poultry housing units.

Access Protocol: Open Access under IEEE Dataport standard data terms for academic research and experimentation.

Repository URL: <https://dx.doi.org/10.21227/dpkp-kx84>

IEEE Dataport Archive (DOI): 10.21227/dpkp-kx84

A.7.2. Caged-Egg Facility Ammonia Telemetry (PLOS ONE)

[21]

Primary Reference: Diana María Gutiérrez-Zapata, Luis Fernando Galeano-Vasco, and Mario Fernando Cerón-Muñoz, Semiparametric Modeling of Daily Ammonia Levels in Naturally Ventilated Caged-Egg Facilities, PLOS ONE, Vol. 11, No. 1, pp. 1–12, 2016.

Data Format: Daily and sub-daily observational records comprising airborne ammonia concentrations, internal temperature curves, and relative humidity regimes. These records provide empirical baselines for modeling the heavy-tailed, zero-inflated volatilization patterns of ammonia gas in livestock environments.

Access Protocol: Open Access distributed under the terms of the Creative Commons Attribution License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Repository URL: <https://doi.org/10.1371/journal.pone.0147135>

Article Archive (DOI): 10.1371/journal.pone.0147135

A.7.3. Dataset Preprocessing and Merge Notes

Feature Alignment: The raw time-series feeds from the repositories above were consolidated and resampled into a unified analytical dataframe totaling 9,708 distinct observations without cleaning data.

Target Variables: The finalized dataset isolates three primary continuous targets for predictive evaluation: Temperature (°C), Relative Humidity (%), and Ammonia Concentration (ppm).

Feature Engineering: To support the Edge-AI Multi-Layer Perceptron (MLP) architecture and autoregressive evaluation, additional temporal variables were derived from the raw data, including 3-step rolling means (roll_mean3), rolling standard deviations (roll_std3), and sequential historical time lags (lag1, lag2, lag3).

REFERENCES

- [1] M. Li, Z. Zhou, Q. Zhang, J. Zhang, Y. Suo, J. Liu, D. Shen, L. Luo, Y. Li, and C. Li, “Multivariate analysis for data mining to characterize poultry house environment in winter,” *Poultry Science*, vol. 103, no. 5, p. 103633, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0032579124002128>
- [2] G. da Rocha Balthazar, R. M. F. Silveira, J. T. Aldrigue, and I. J. O. da Silva, “Development and validation of a rapid-prototyping IoT-based sensor system for poultry house microclimate monitoring,” *Smart Agricultural Technology*, vol. 12, p. 101197, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772375525004289>
- [3] S. Surateno, S. Kautsar, R. Rachmanita, A. Anwaludin, M. Adhiyatma, R. Hertawamati, B. Hariono, F. Purnomo, and S. Sarena, “Smart control and monitoring system for closed poultry house based on IoT,” *International Journal of Applied Sciences and Smart Technologies*, vol. 6, pp. 25–40, 06 2024.
- [4] M. Naeem, Z. Jia, J. Wang, S. Poudel, S. Manjankattil, Y. Adhikari, M. Bailey, and D. Bourassa, “Advancements in machine learning applications in poultry farming: a literature review,” *Journal of Applied Poultry Research*, vol. 34, no. 4, p. 100602, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1056617125000868>
- [5] M. A. H. Bappy, M. R. Hoshen, M. M. Rafid, M. M. Hossain, and M. Afroja, “Low-cost internet of things (IoT)-based climate control automation for improved poultry growth and survival in least developing countries,” *Cureus Journal of Engineering*, vol. 2, 2025.
- [6] Cobb-Vantress, *Cobb 500 Broiler Performance & Nutrition Supplement*, 2022, accessed: 2026-06-14. [Online]. Available: <https://www.cobbgenetics.com/assets/Cobb-Files/2022-Cobb500-Broiler-Performance-Nutrition-Supplement.pdf>
- [7] Aviagen, *Ross Broiler Management Handbook*, 2025, accessed: 2026-06-14. [Online]. Available: https://aviagen.com/assets/Tech_Center/Ross_Broiler/Aviagen-ROSS-Broiler-Handbook-EN.pdf
- [8] H. Lin, H. Jiao, J. Buyse, and E. Decuypere, “Strategies for preventing heat stress in poultry,” *Worlds Poultry Science Journal - WORLD POULTRY SCI J*, vol. 62, pp. 71–85, 03 2006.
- [9] H. Lashari, A. Memon, S. Shah, K. Nenwani, and F. Shafqat, “IoT based poultry environment monitoring system,” in *2018 IEEE International Conference on*

Internet of Things and Applications (IOTAIS), 11 2018, pp. 1–5.

- [10] W. D. Weaver and R. Meijerhof, “The effect of different levels of relative humidity and air movement on litter conditions, ammonia levels, growth, and carcass quality for broiler chickens,” *Poultry Science*, vol. 70, no. 4, pp. 746–755, 1991.
- [11] S. I. Orakwue, H. M. R. Al-Khafaji, and M. Chabuk, “IoT based smart monitoring system for efficient poultry farming,” *Webology*, vol. 19, pp. 4105–4112, 01 2022.
- [12] W. Y. Leong, “Smart poultry farming 4.0: Leveraging IoT for climate control and productivity,” in *2025 21st IEEE International Colloquium on Signal Processing & Its Applications (CSPA)*, 03 2025, pp. 172–175.
- [13] D. Cassuce, I. Tinôco, F. Baêta, S. Zolnier, P. Cecon, and M. d. F. Vieira, “Thermal comfort temperature update for broiler chickens up to 21 days of age,” *Engenharia Agrícola*, vol. 33, pp. 28–36, 02 2013.
- [14] M. Yerpes, P. Llonch, and X. Manteca, “Factors associated with cumulative first-week mortality in broiler chicks,” *Animals*, vol. 10, p. 310, 02 2020.
- [15] L. Barbosa, N. Lima, J. Barros, D. Moura, F. Estellés, A. Ramón-Moragues, S. Calvet-Sanz, and A. García, “Predicting risk of ammonia exposure in broiler housing: Correlation with incidence of health issues,” *Animals*, vol. 14, p. 615, 02 2024.
- [16] H. Han, Y. Zhou, Q. Liu, G. Wang, J. Feng, and M. Zhang, “Effects of ammonia on gut microbiota and growth performance of broiler chickens,” *Animals*, vol. 11, p. 1716, 06 2021.
- [17] L. M. Parandy, S. S. Hidayat, M. Robby, R. Azhar, I. Mujahidin, and M. Prabowo, “Design of temperature monitoring and control system in smart hen coop based on internet of things,” *Journal of Information System, Technology and Engineering*, vol. 2, pp. 236–251, 06 2024.
- [18] D. Miles, S. Branton, and B. Lott, “Atmospheric ammonia is detrimental to the performance of modern commercial broilers,” *Poultry Science*, vol. 83, no. 10, pp. 1650–1654, 2004. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0032579119444568>
- [19] G. Quintana-Ospina, M. Alfaro-Wisaquillo, E. Oviedo, J. Ruiz-Ramirez, L. Bernal-Arango, and G. Martinez-Bernal, “Effect of environmental and farm-associated factors on live performance parameters of broilers raised under commercial tropical conditions,” *Animals*, vol. 13, p. 3312, 10 2023.

- [20] A. Barsagade, “Poultry farm environmental sensor dataset,” 2023. [Online]. Available: <https://dx.doi.org/10.21227/dpkp-kx84>
- [21] D. M. Gutiérrez-Zapata, L. F. Galeano-Vasco, and M. F. Cerón-Muñoz, “Semiparametric modeling of daily ammonia levels in naturally ventilated caged-egg facilities,” *PLOS ONE*, vol. 11, pp. 1–12, 01 2016. [Online]. Available: <https://doi.org/10.1371/journal.pone.0147135>
- [22] E. Küçüktopçu, B. Cemek, and H. Simsek, “Machine learning and wavelet transform: A hybrid approach to predicting ammonia levels in poultry farms,” *Animals*, vol. 14, no. 20, 2024. [Online]. Available: <https://www.mdpi.com/2076-2615/14/20/2951>
- [23] Espressif Systems, “ESP32,” 2024, accessed: 2026-08-24. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>
- [24] —, “ESP32-S3,” 2024, accessed: 2026-08-24. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32-s3>
- [25] ESP Boards, “DOIT ESP32 DEVKIT V1 Development Board Pinout and Technical Specifications,” 2023, accessed: 2026-08-24. [Online]. Available: <https://www.espboards.dev/esp32/esp32doit-devkit-v1/>
- [26] Espressif Systems, “ESP32-S3-DevKitC-1,” 2024, accessed: 2026-08-24. [Online]. Available: <https://www.espressif.com/en/dev-board/esp32-s3-devkitc-1-en>
- [27] Zhengzhou Winsen Electronics Technology Co., Ltd., *MQ137 Ammonia Gas Sensor*, 07 2021, version 1.6.
- [28] Robu.in, “MQ-137 NH3 Ammonia Gas Sensor Module,” 2023, accessed: 2026-08-24. [Online]. Available: <https://robu.in/product/mq-137-ammonia-gas-nh3-sensor-module/>
- [29] Guangzhou Aosong Electronics Co., Ltd., *AM2302 Technical Manual*, ASAIR.
- [30] Components101, “DHT22 Sensor Pinout, Specs, Equivalents, Circuit & Datasheet,” 2023, accessed: 2026-08-24. [Online]. Available: <https://components101.com/sensors/dht22-pinout-specs-datasheet>
- [31] Texas Instruments, *LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator*, 03 2023, datasheet SNVS124G.
- [32] Python Software Foundation, “The python standard library — python 3 documentation,” 2026, accessed: 2026-08-25. [Online]. Available: <https://docs.python.org/3/>

[//docs.python.org/3](https://docs.python.org/3)

- [33] M. Abadi, A. Agarwal, P. Barham *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org/>
- [34] F. Chollet *et al.*, “Keras,” 2015. [Online]. Available: <https://keras.io>
- [35] The pandas development team, “pandas-dev/pandas: Pandas,” Feb. 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.3509134>
- [36] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [37] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [38] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, “TensorFlow lite micro: Embedded machine learning on TinyML systems,” 2020. [Online]. Available: <https://arxiv.org/abs/2010.08678>
- [39] PlatformIO, “What is PlatformIO?” 2026, accessed: 2026-08-25. [Online]. Available: <https://docs.platformio.org/en/latest/what-is-platformio.html>